Graph Traversal Algorithm

- A graph traversal algorithm tries to visit all the nodes it can reach.

- If a graph is disconnected, a graph traversal that begins at a node **v** will visit only a subset of nodes, that is, the connected component containing **v**.

Traversal algorithms

Two basic graph traversal algorithms:

- **Depth-first-search (DFS)**
  - After visiting node **v**, DFS strategy proceeds along a path from **v** as deeply into the graph as possible before backtracking.

- **Breadth-first-search (BFS)**
  - After visiting node **v**, BFS strategy visits every node adjacent to **v** before visiting any other nodes.

Depth-First Search

- A DFS traversal of a graph G
  - Visits all the vertices and edges of G
  - Determines whether G is connected
  - Computes the connected components of G
  - Computes a spanning forest of G

- DFS on a graph with n vertices and m edges takes $O(n + m)$ time

- DFS can be further extended to solve other graph problems
  - Find and report a path between two given vertices
  - Find a cycle in the graph

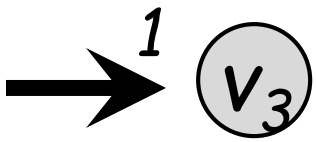
Depth-first search (DFS)

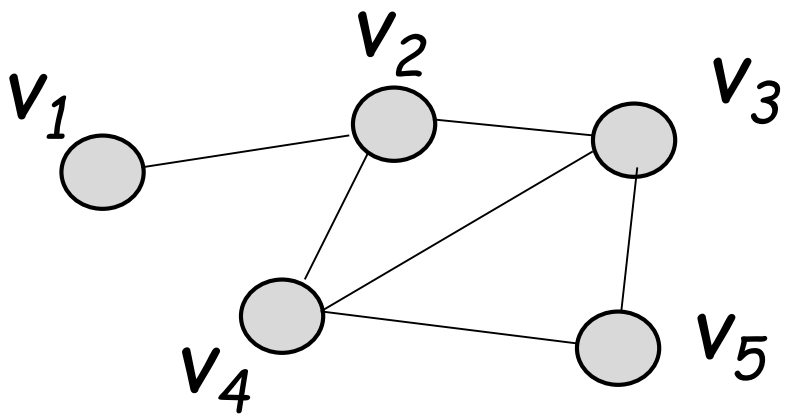
- From a given node v, it first visits itself. Then, recursively visit its unvisited neighbours one by one.

```
Algorithm dfs(v)
    mark v as visited ;
    for (each unvisited node u adjacent to v)
       dfs(u) ;
```

DFS example

Start from v_3

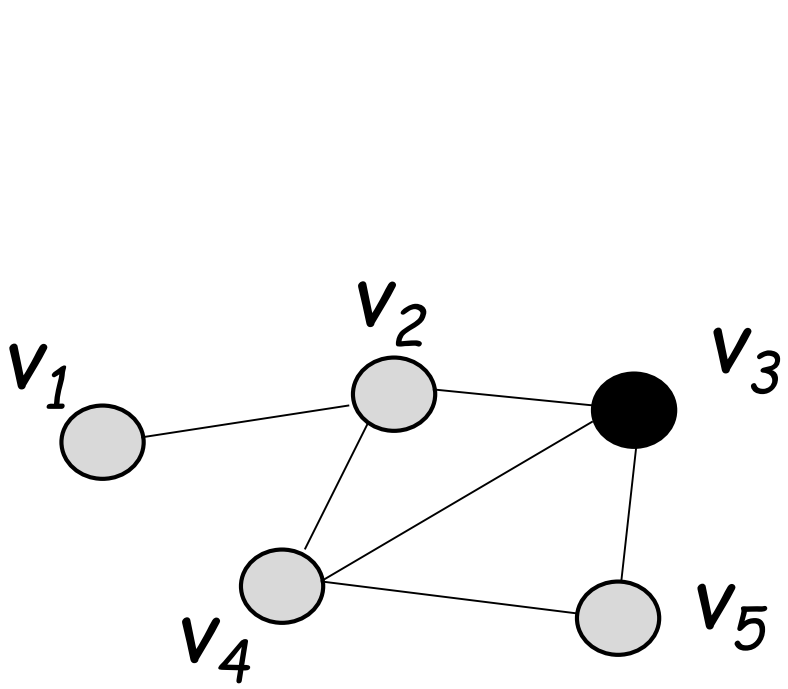




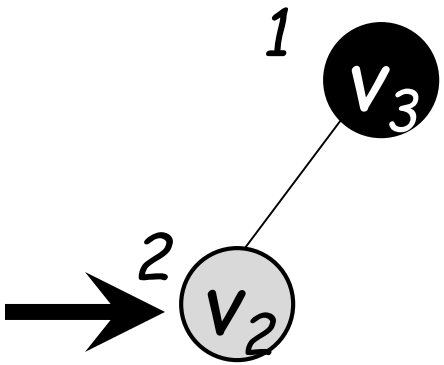
G

DFS example

Start from v_3

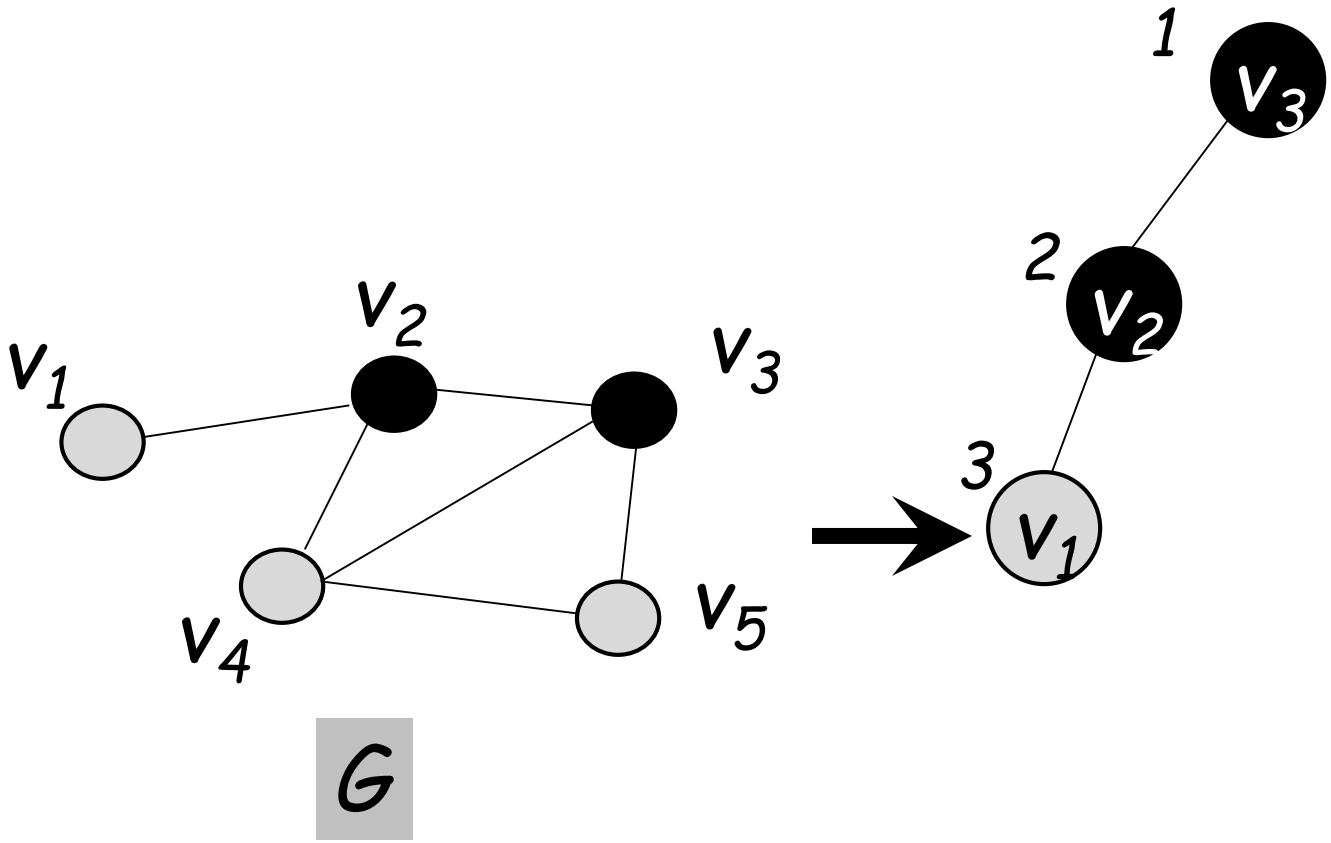


G



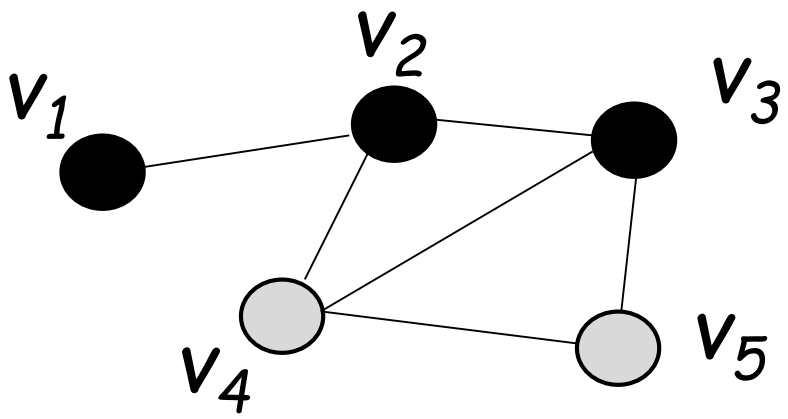
DFS example

Start from v_3

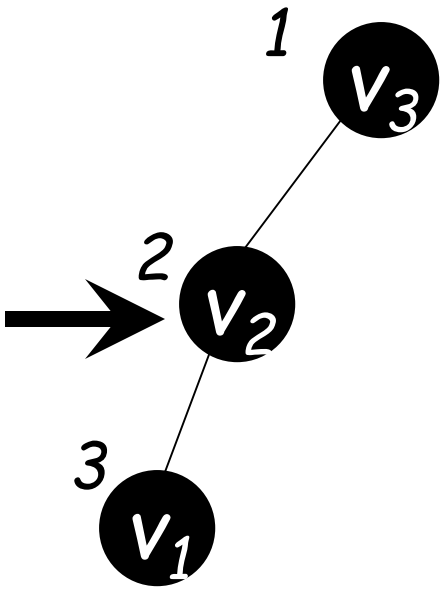


DFS example

Start from v_3

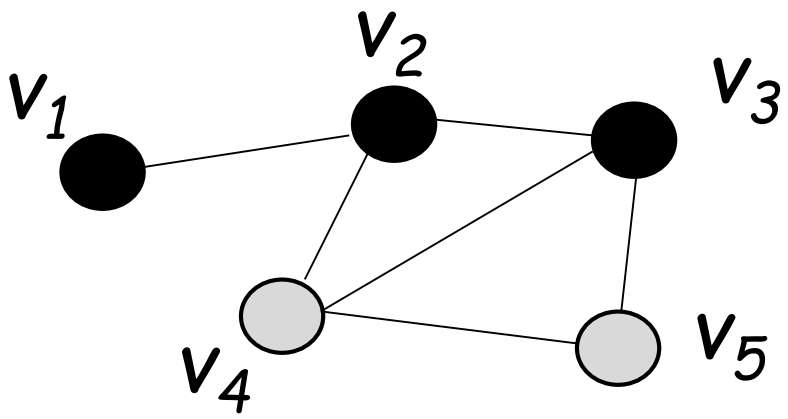


G

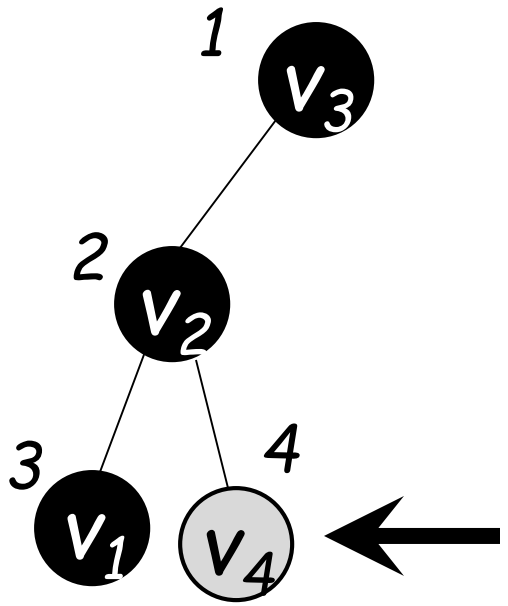


DFS example

Start from v_3

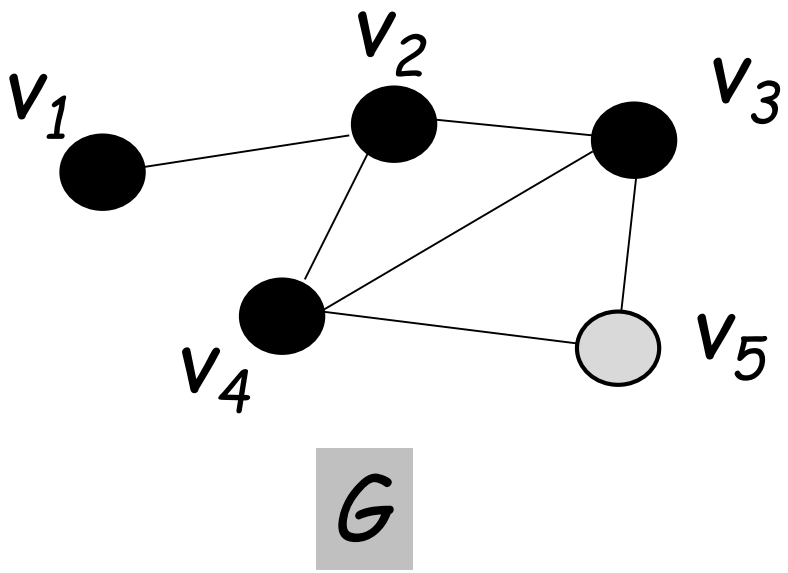


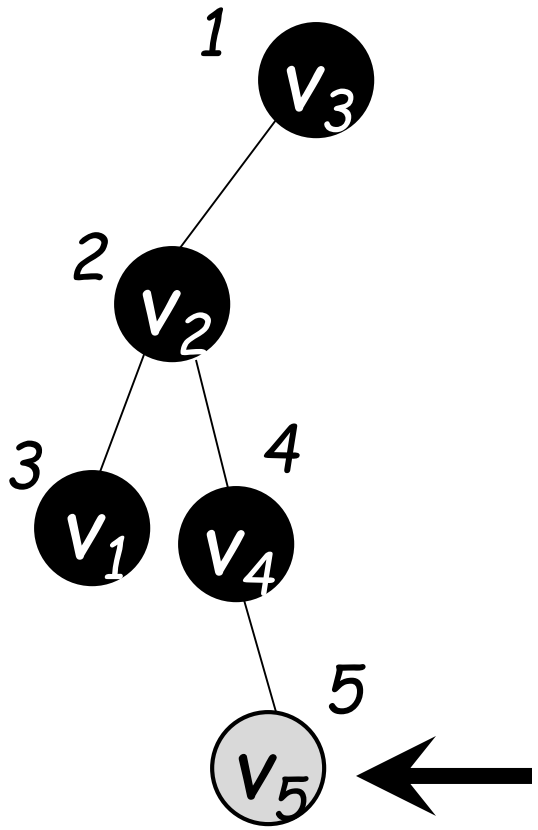
G



DFS example

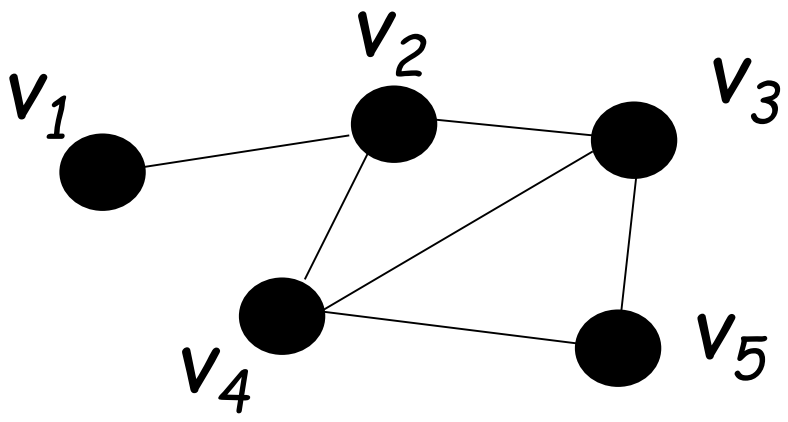
Start from v_3



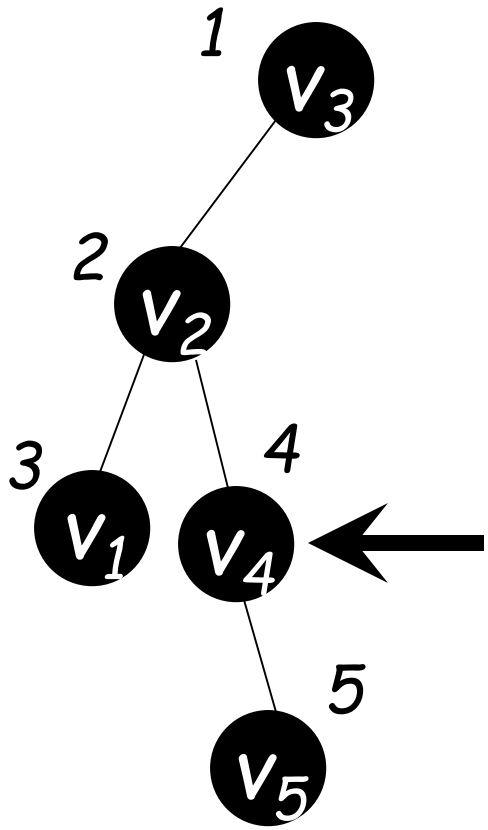


DFS example

Start from v_3

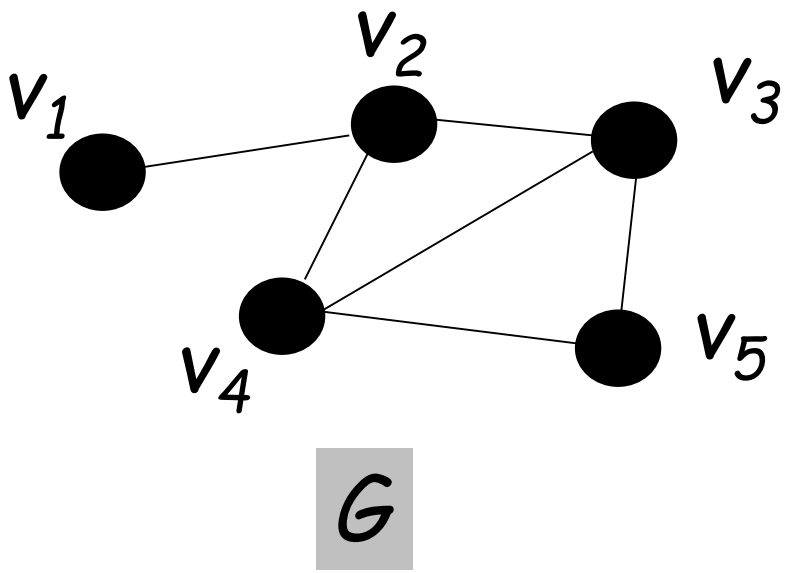


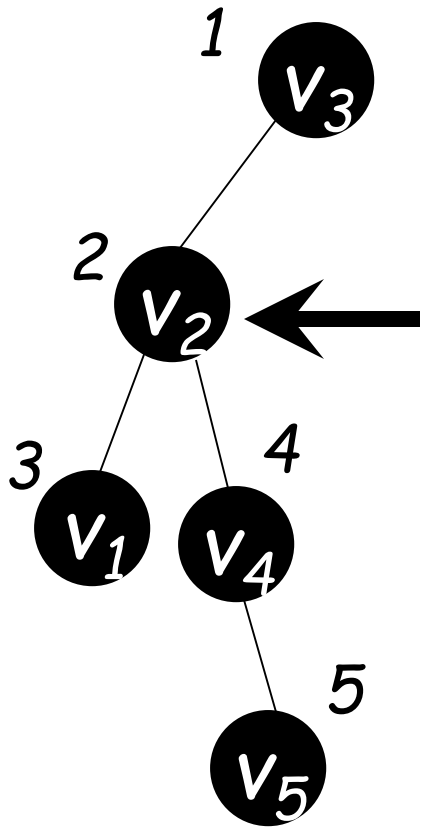
G



DFS example

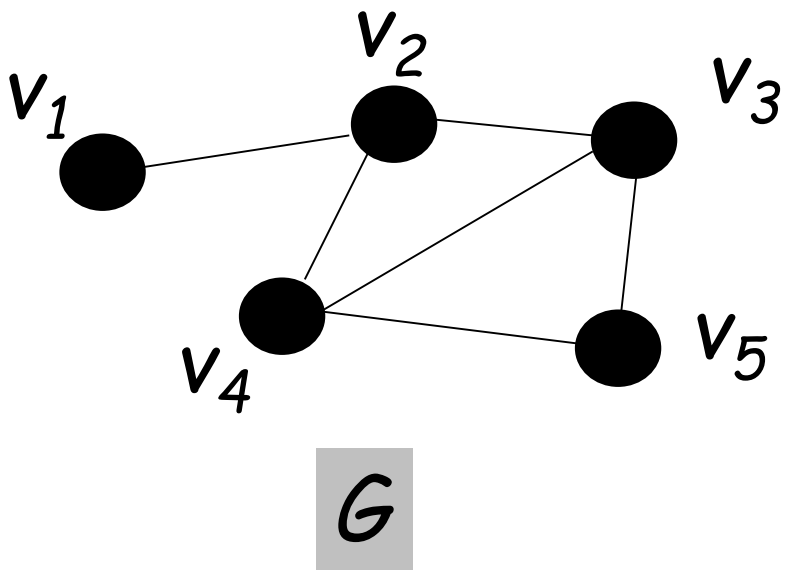
Start from v_3

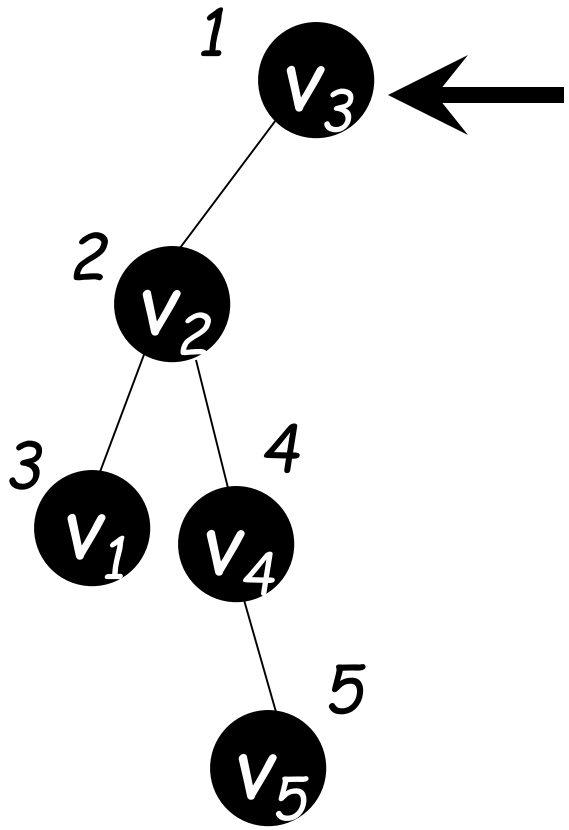




DFS example

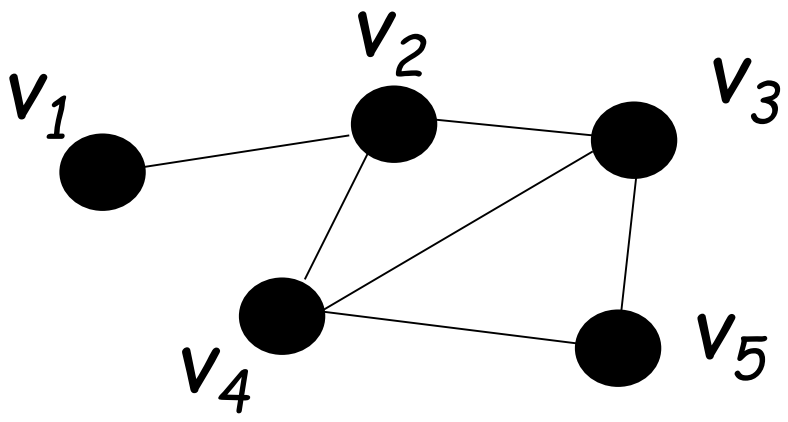
Start from v_3



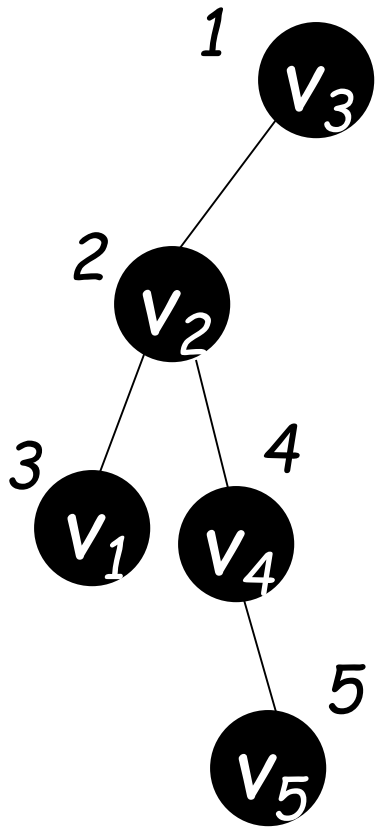


DFS example

Start from v_3



G

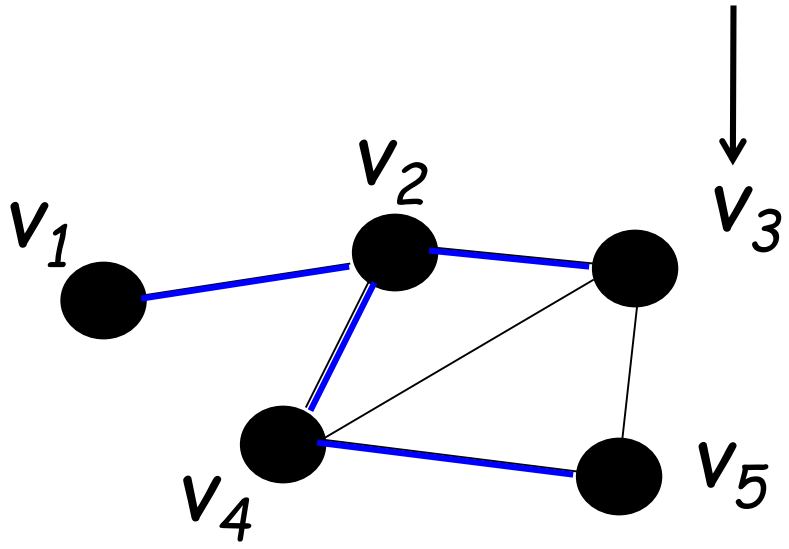


Non-recursive version of DFS algorithm

```
Algorithm dfs(v)
    s.createStack();
    s.push(v);
    mark v as visited;
    while (!s.isEmpty()) {
       let x be the node on top of the stack s;
       if (no unvisited nodes are adjacent to x)
         s.pop() ;  // backtrack
       else {
         select an unvisited node u adjacent to x;
         s.push(u);
         mark u as visited;
       }
    }
```

Non-recursive DFS example

visit	stack
v_3	v_3
v_2	v_3, v_2
v_1	v_3, v_2, v_1
backtrack	v_3, v_2
v_4	v_3, v_2, v_4
v_5	v_3, v_2, v_4, v_5
backtrack	v_3, v_2, v_4
backtrack	v_3, v_2
backtrack	v_3
backtrack	empty



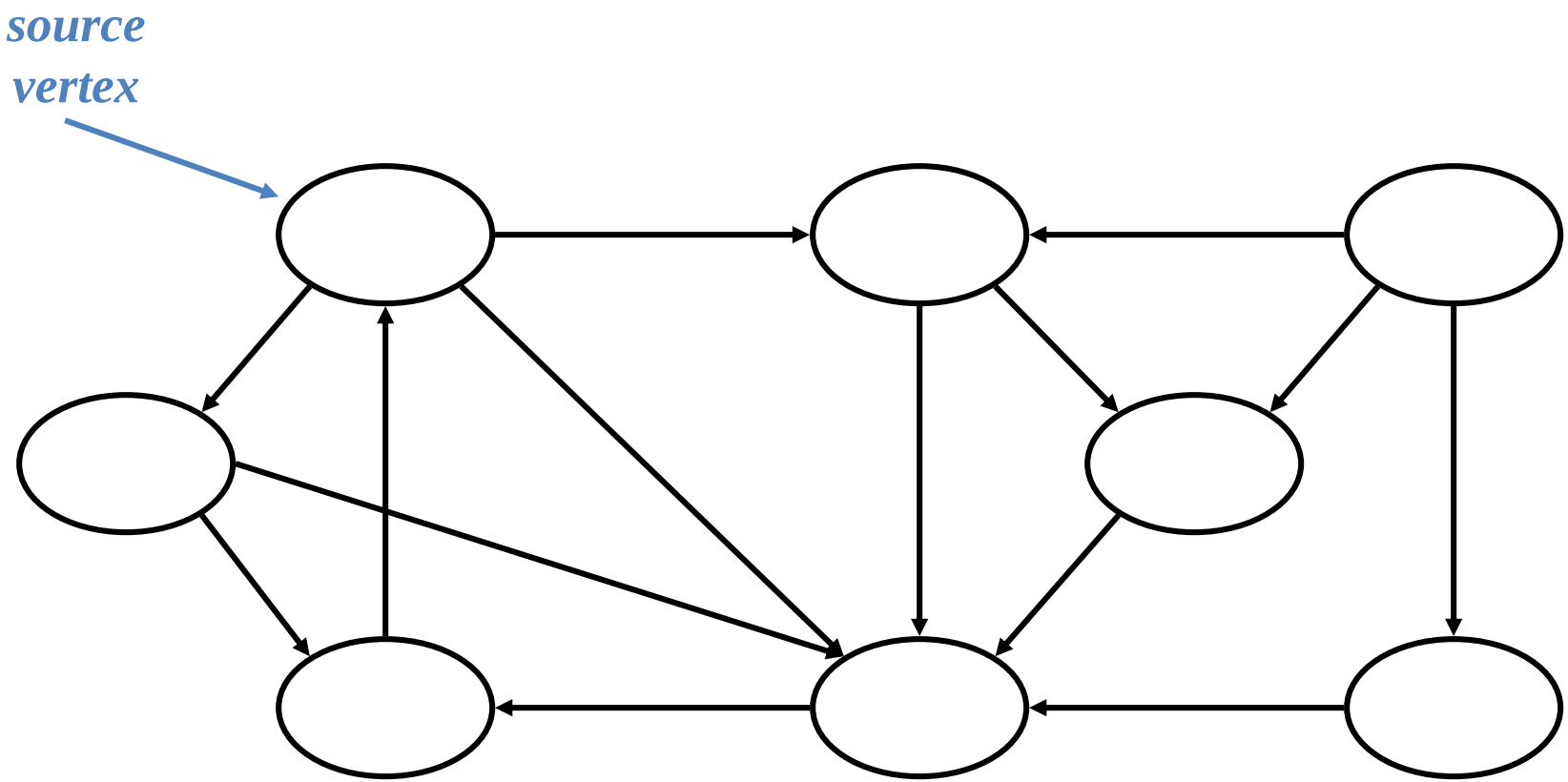
G

Depth-First Search: The Code

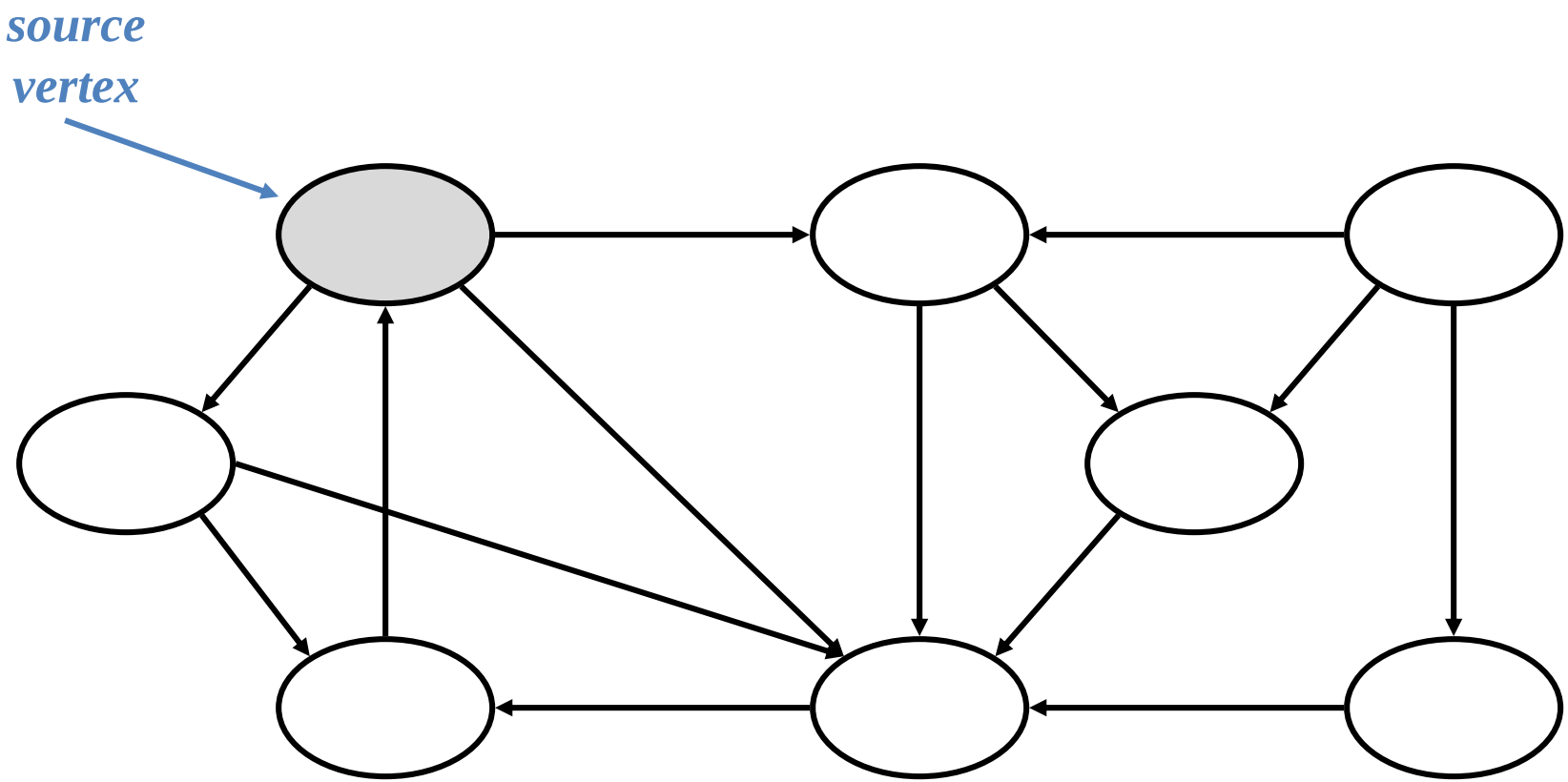
```
DFS(G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
   for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
   u->color = GREY;
   for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
}
```

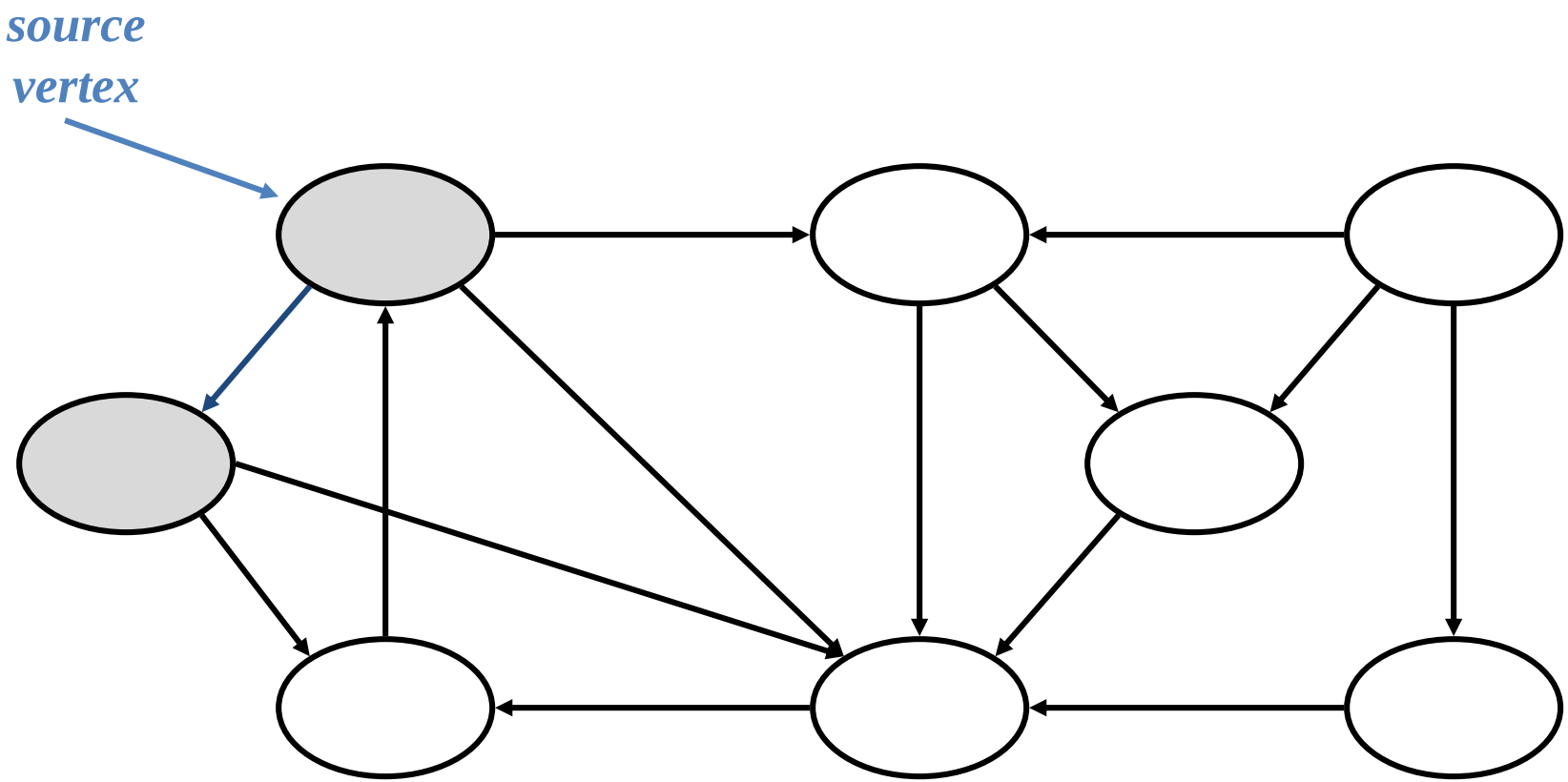
DFS Example



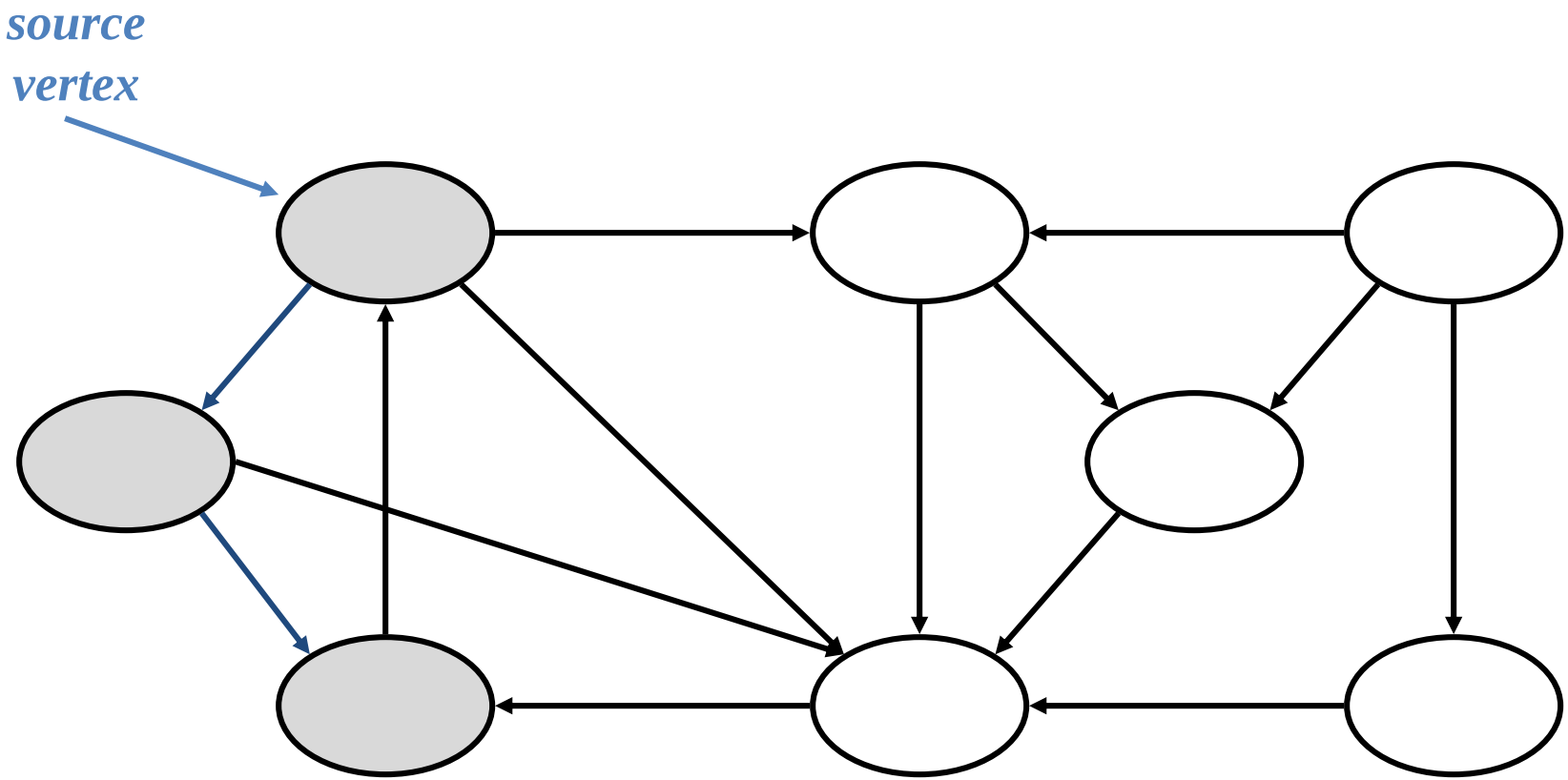
DFS Example



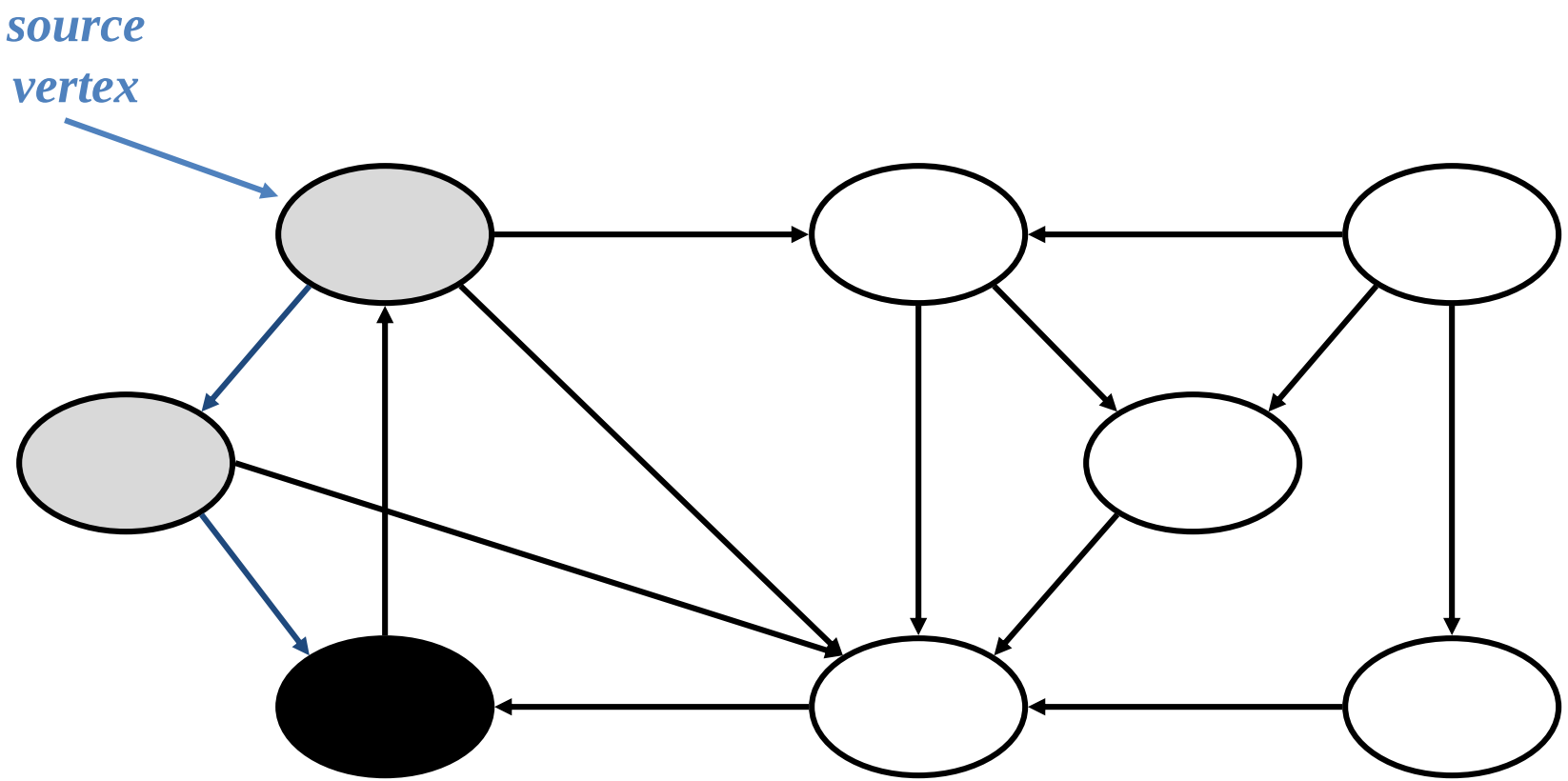
DFS Example



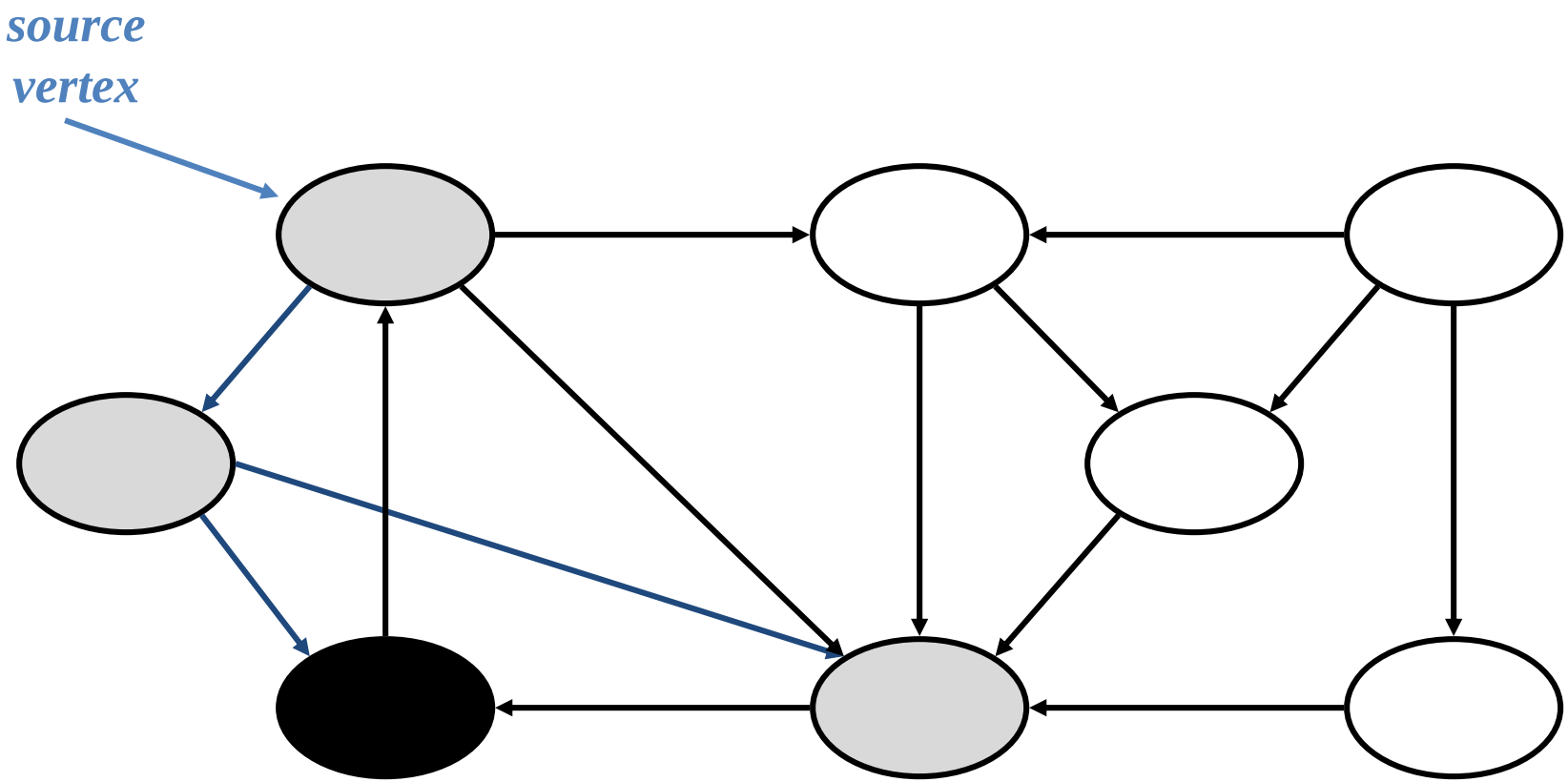
DFS Example



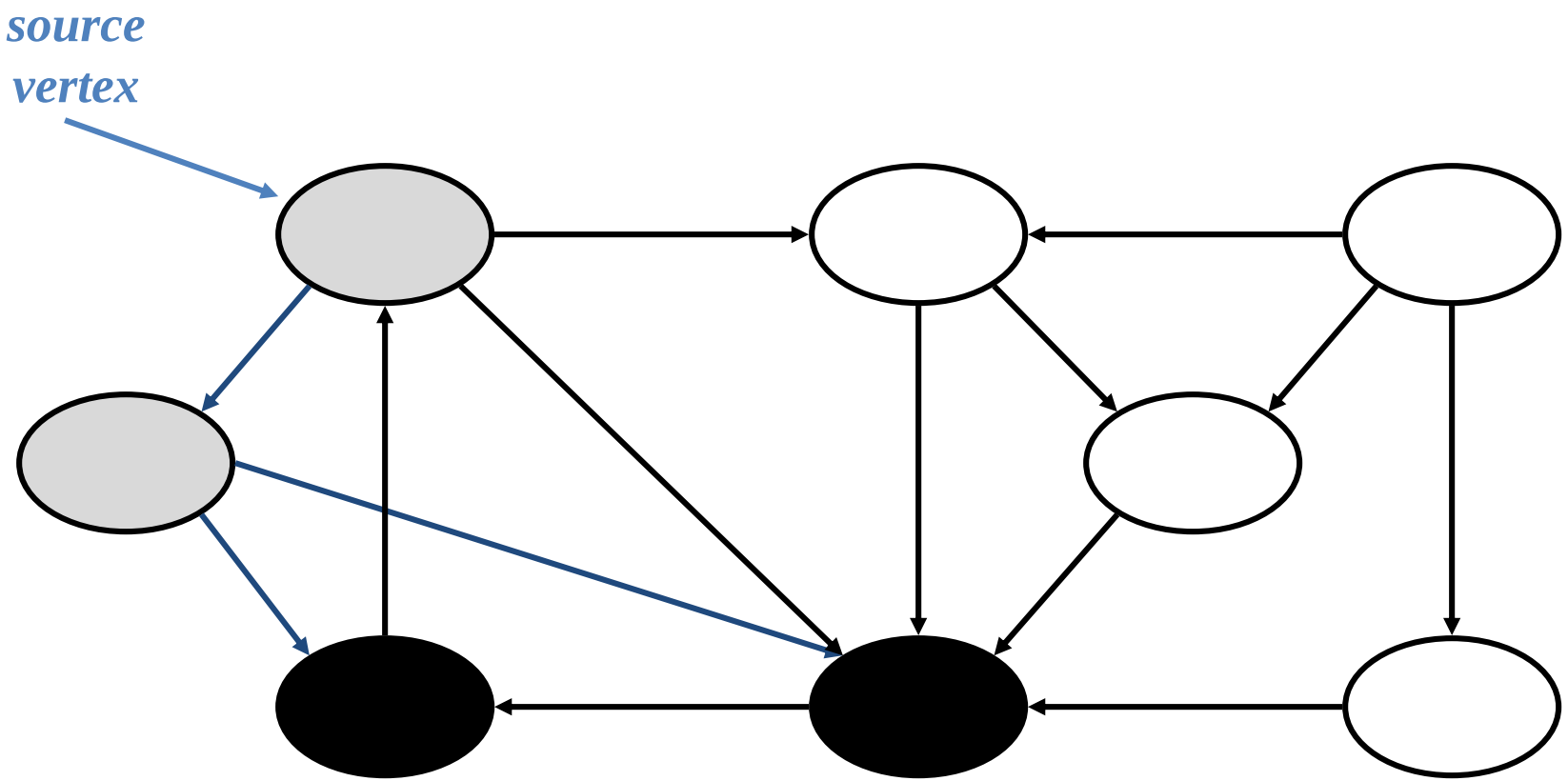
DFS Example



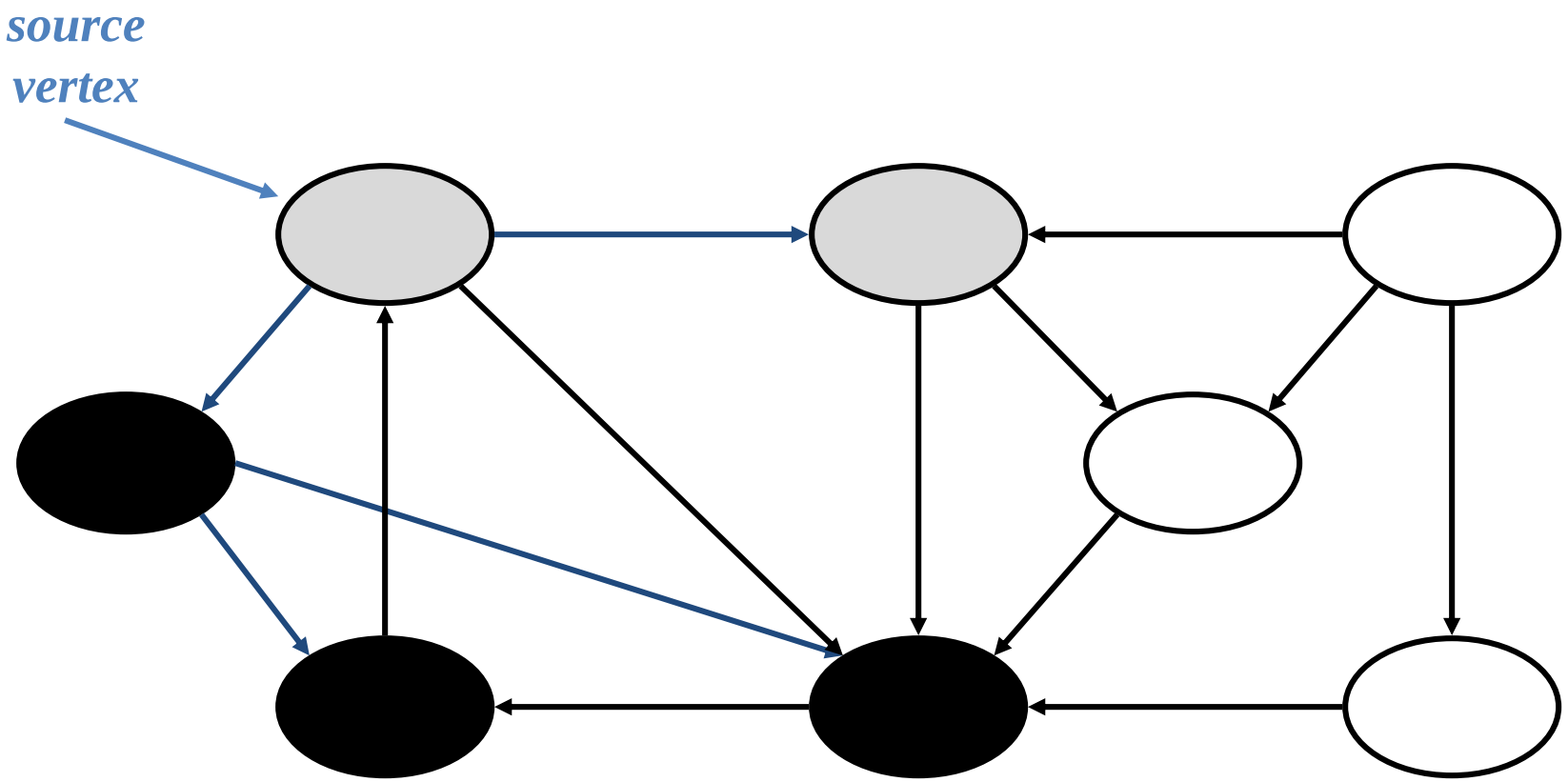
DFS Example



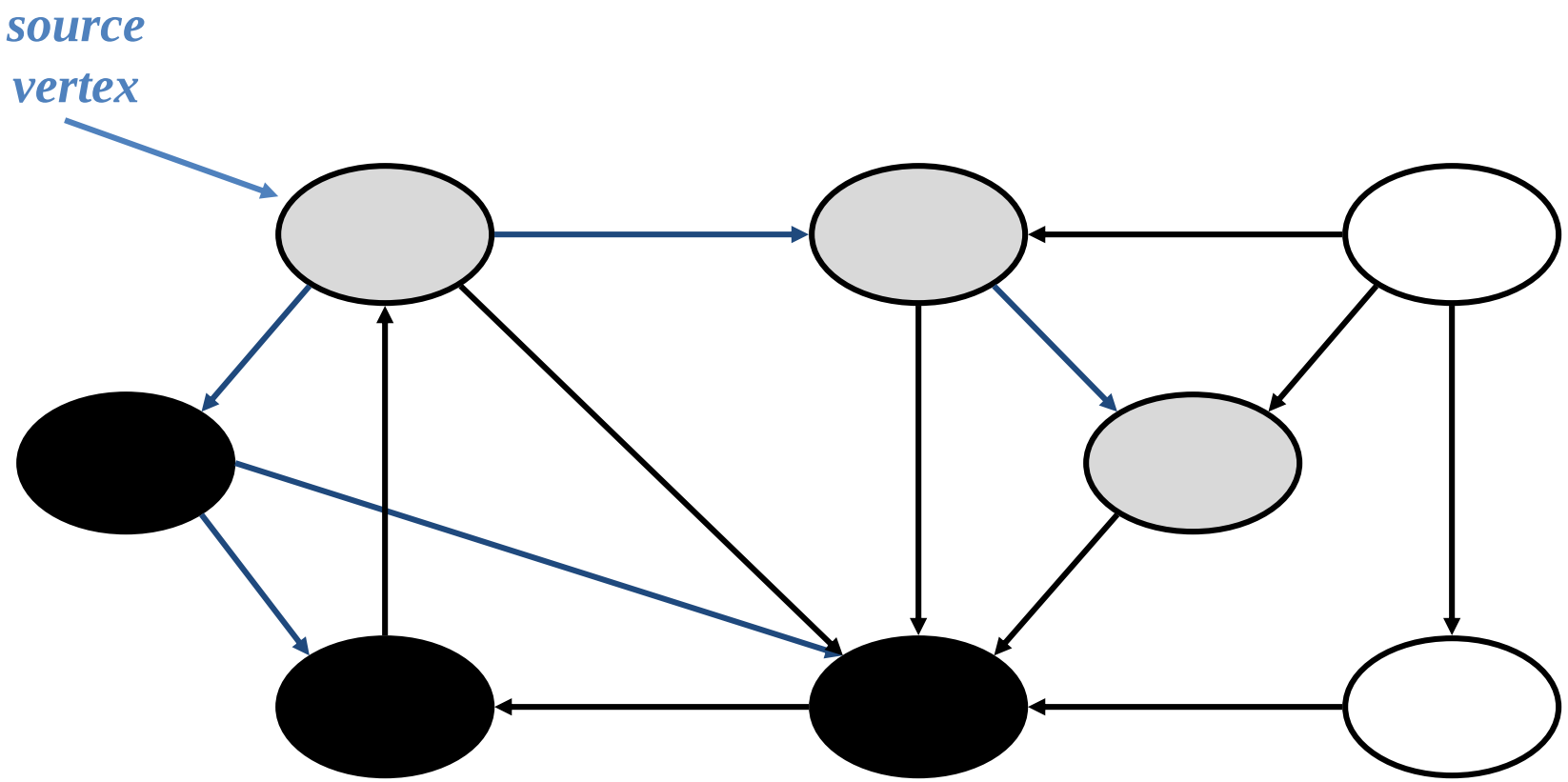
DFS Example



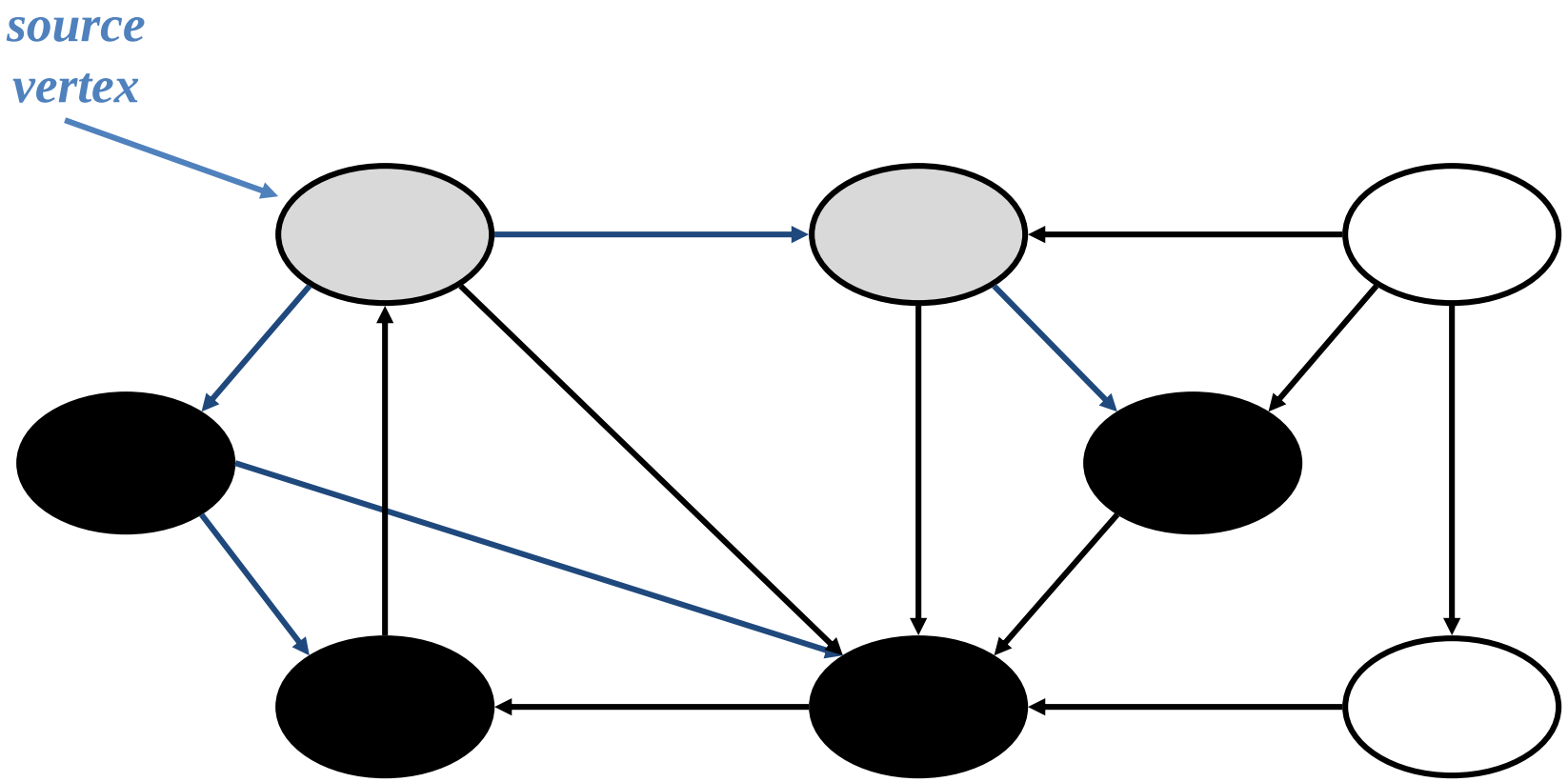
DFS Example



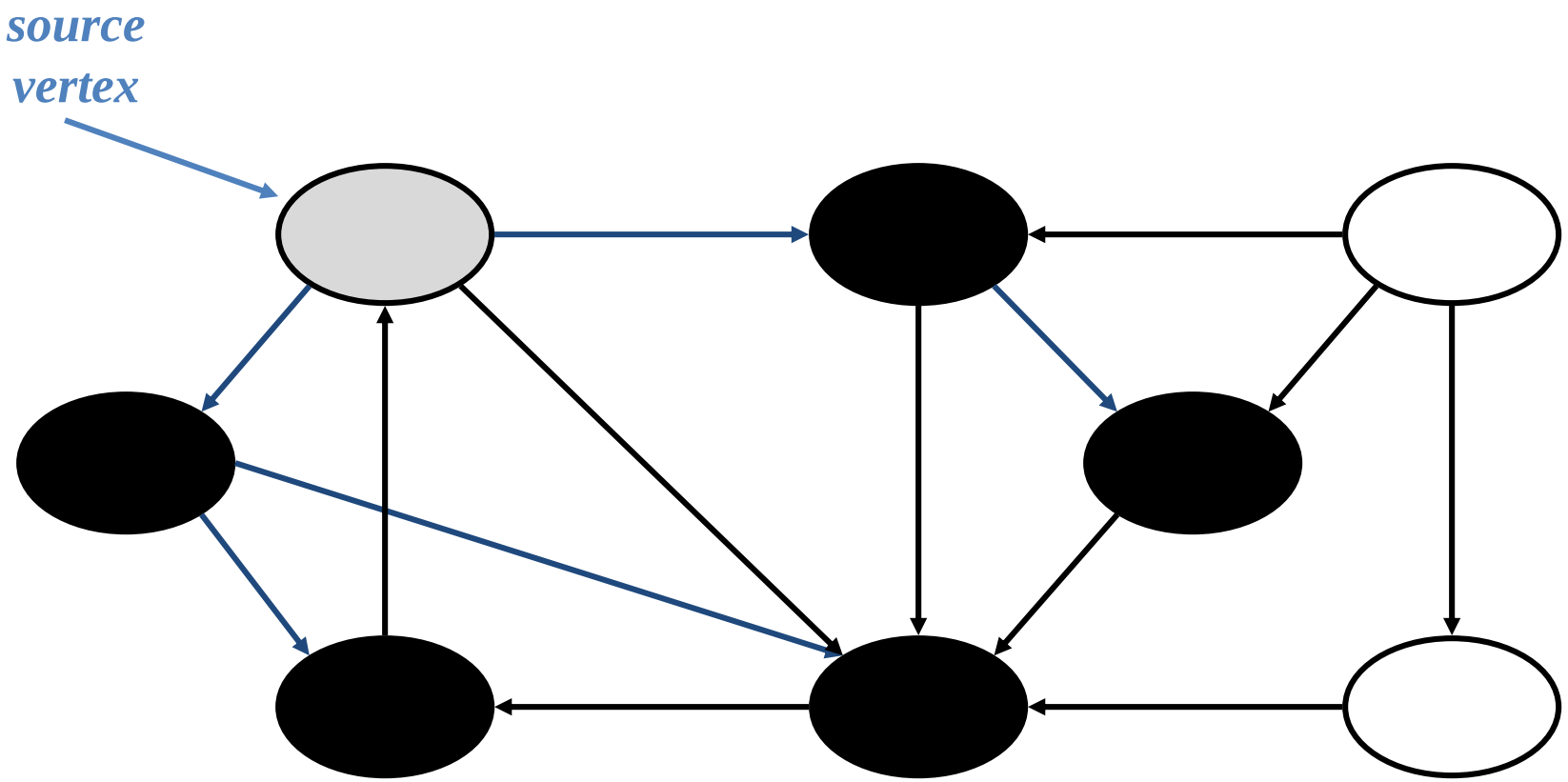
DFS Example



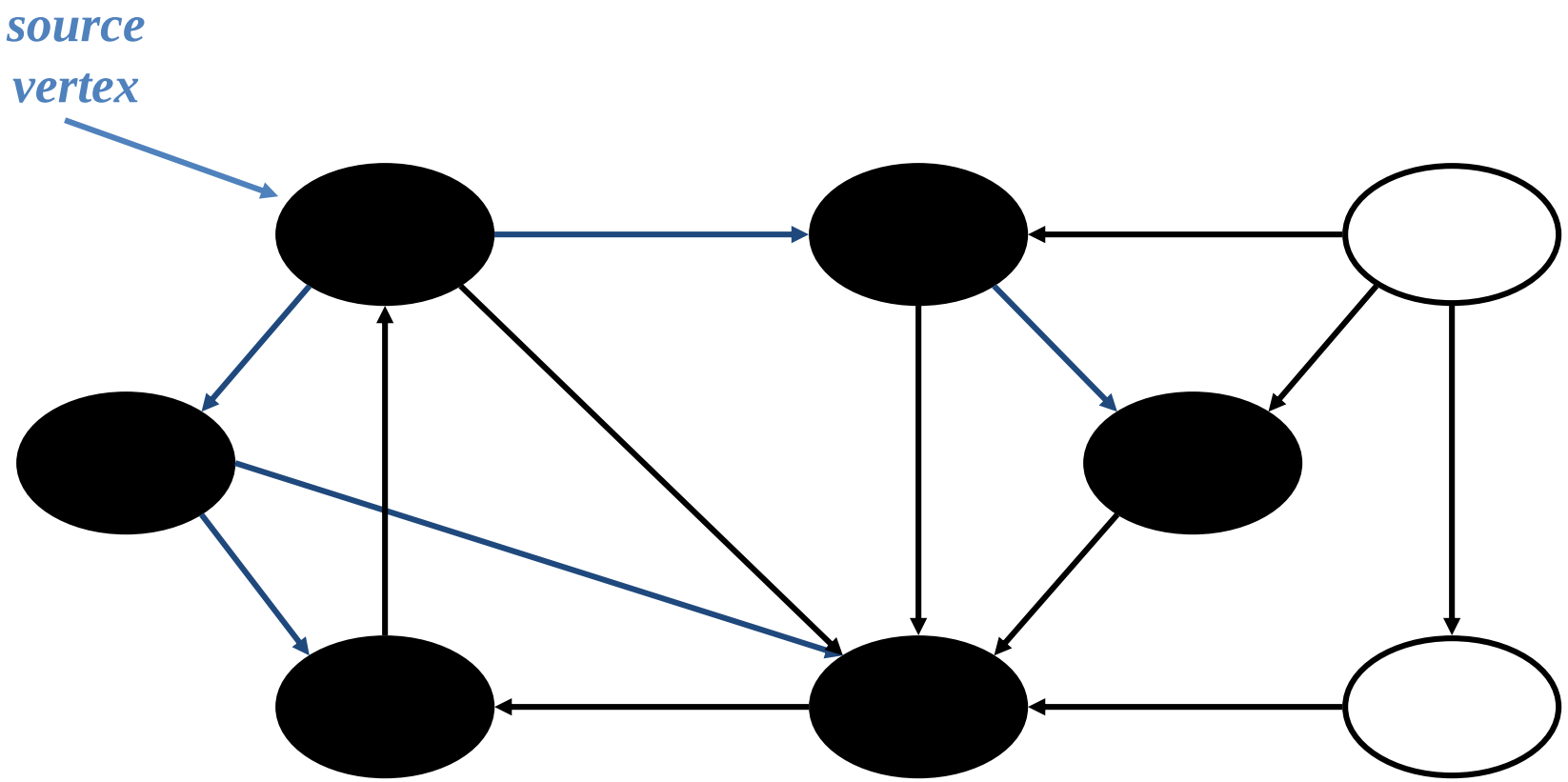
DFS Example



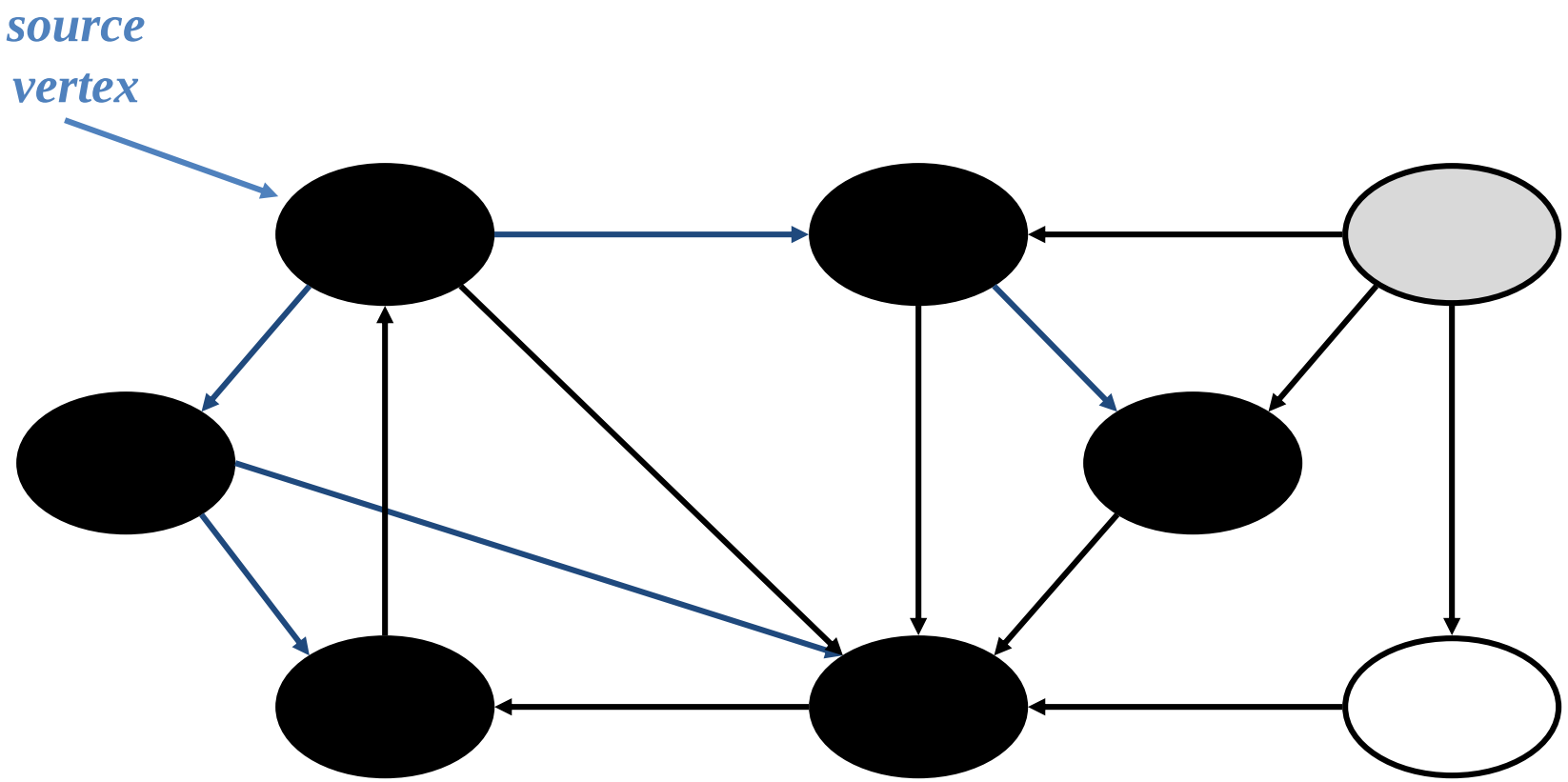
DFS Example



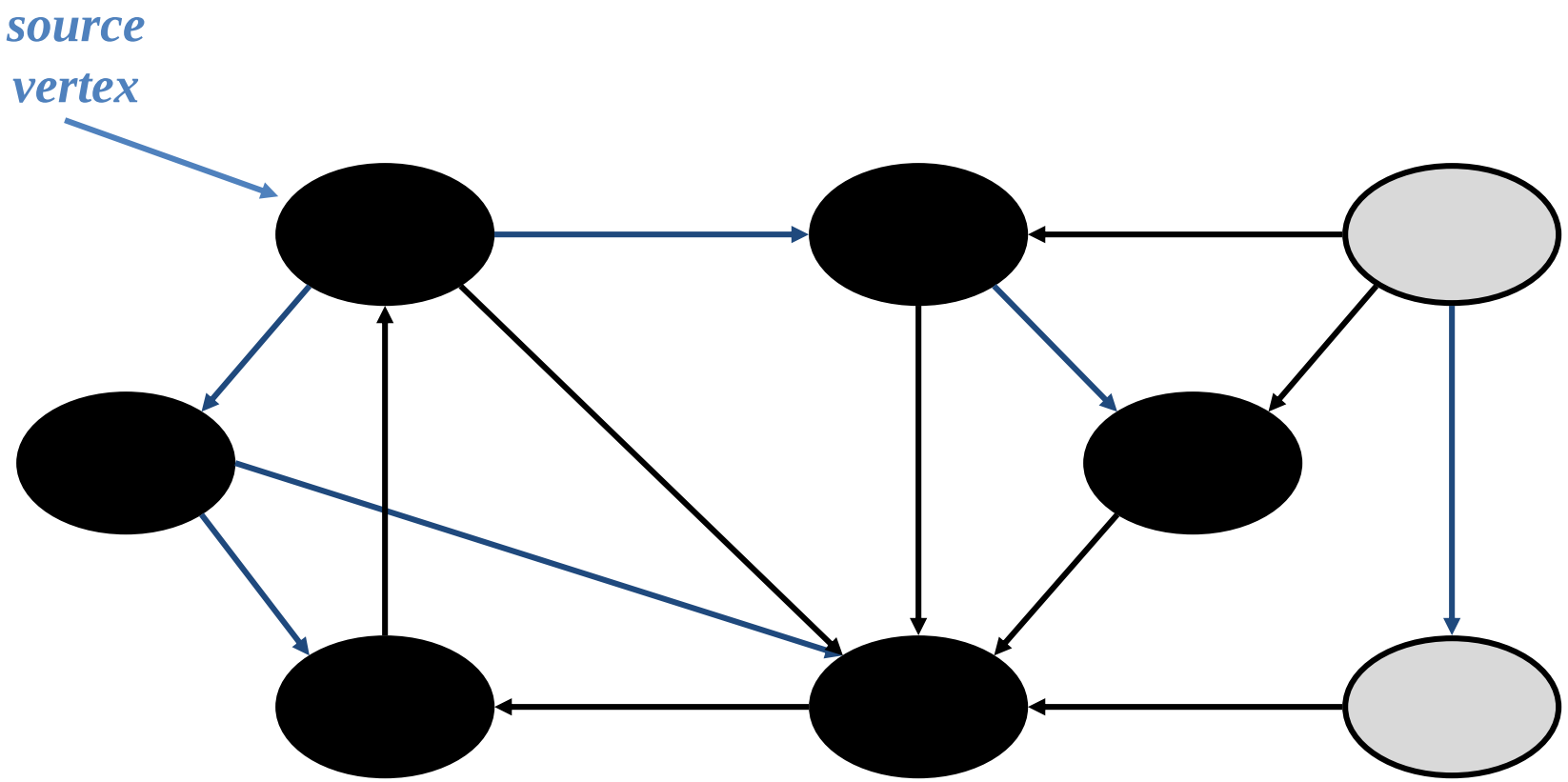
DFS Example



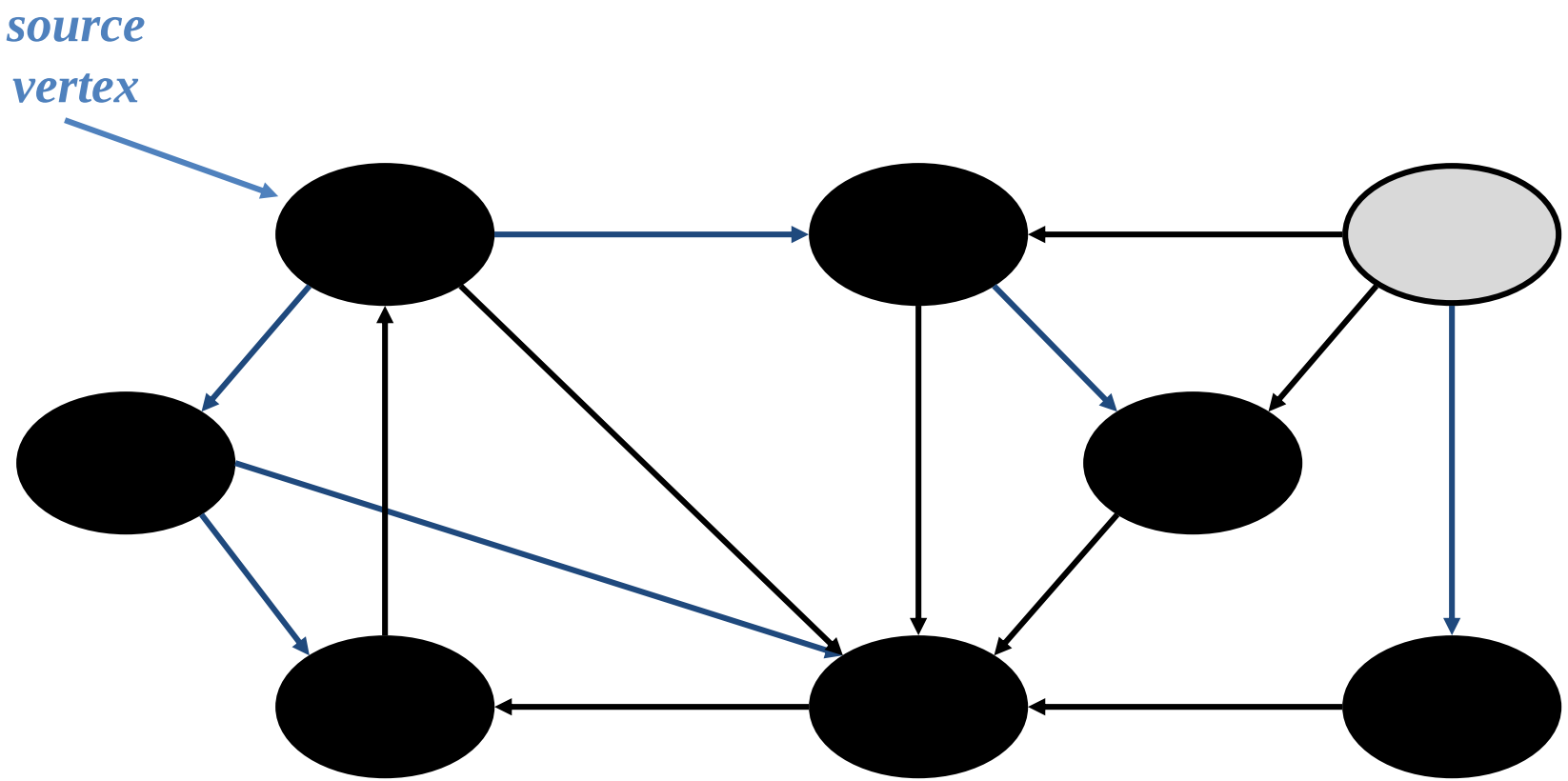
DFS Example



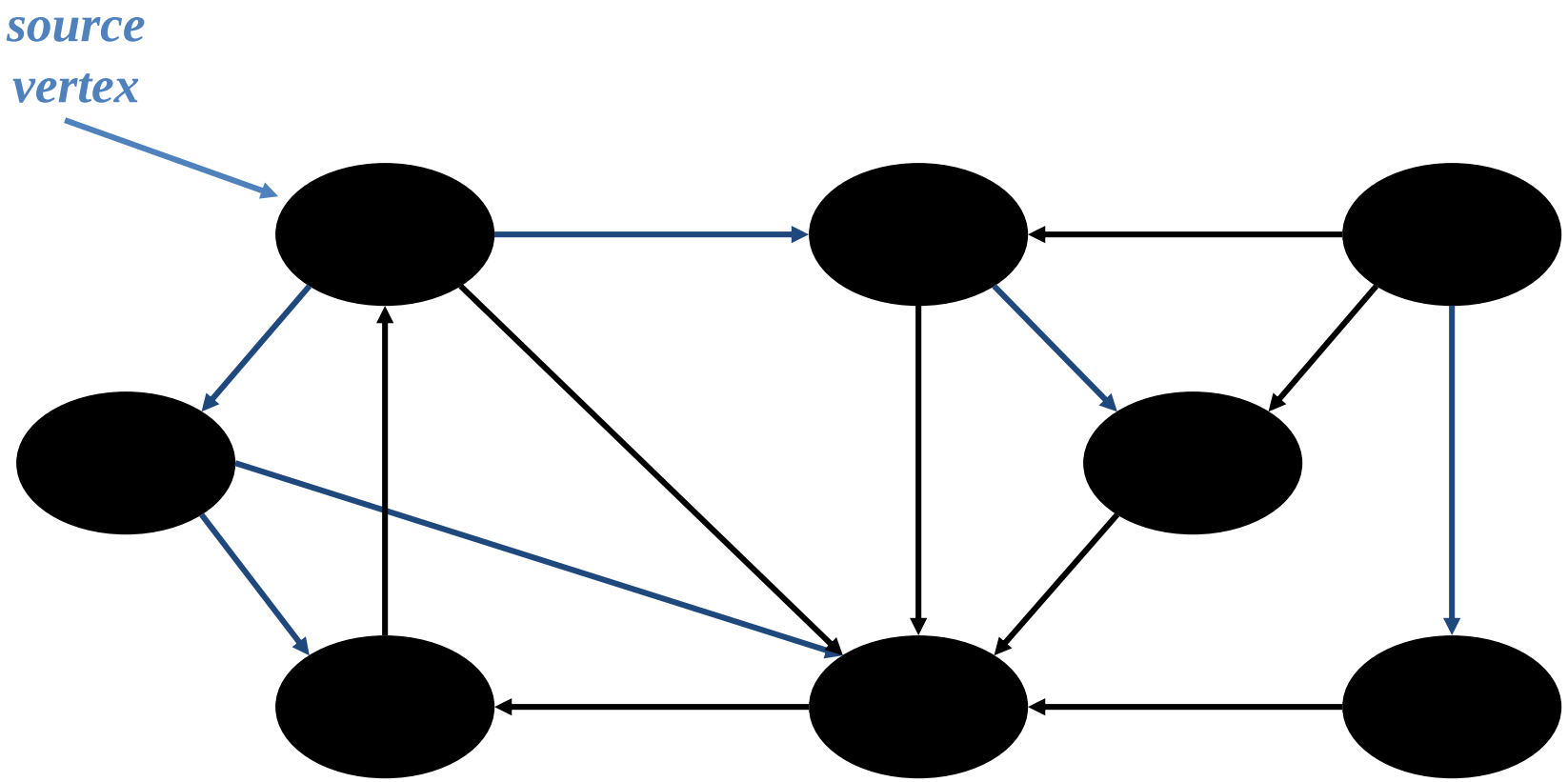
DFS Example



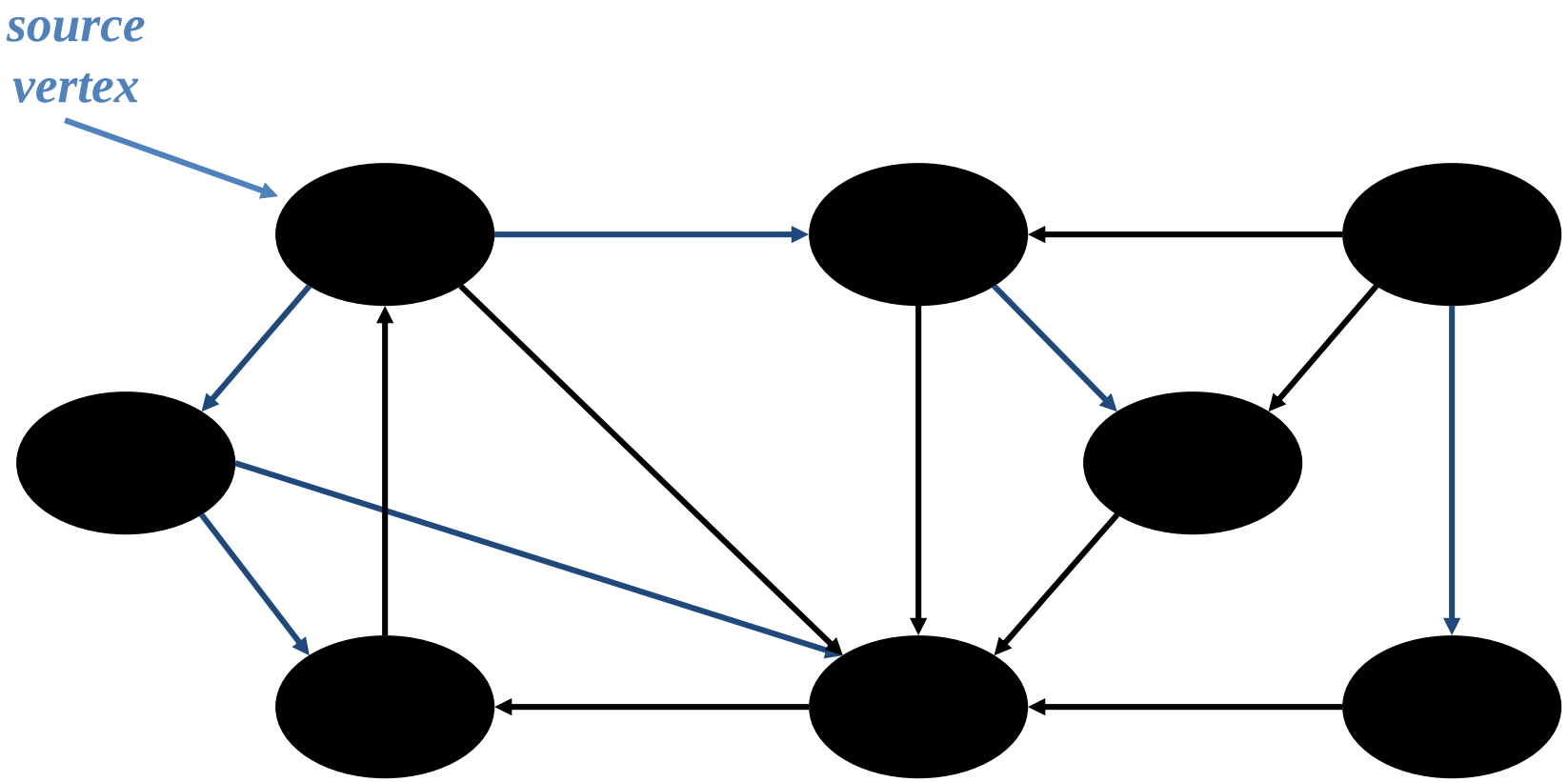
DFS Example



DFS Example



DFS Example



Breadth-First Search

- A BFS traversal of a graph G
  - Visits all the vertices and edges of G
  - Determines whether G is connected
  - Computes the connected components of G
  - Computes a spanning forest of G

- BFS on a graph with n vertices and m edges takes $O(n + m)$ time

- BFS can be further extended to solve other graph problems
  - Find a path with the minimum number of edges between two given vertices.
  - Find a simple cycle, if there is one.

Breadth-first search (BFS)

From a given node **v**, it first visits itself. Then, it visits every node adjacent to **v** before visiting any other nodes.

1. Visit **v**
2. Visit all **v**'s neigbours
3. Visit all **v**'s neighbours' neighbours

.

.

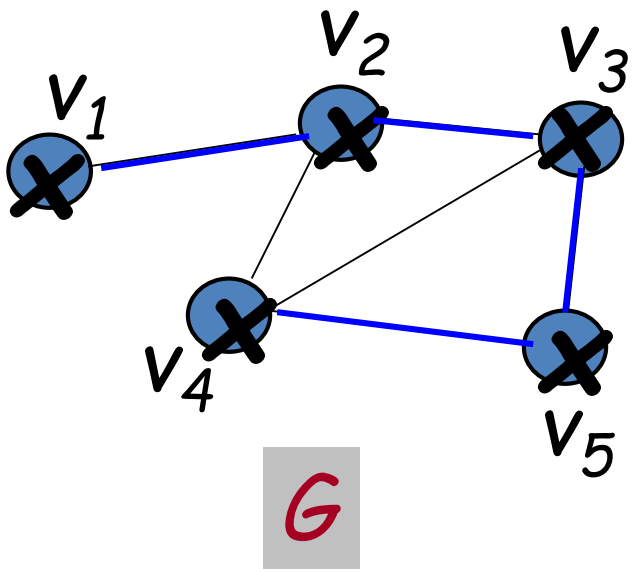
.

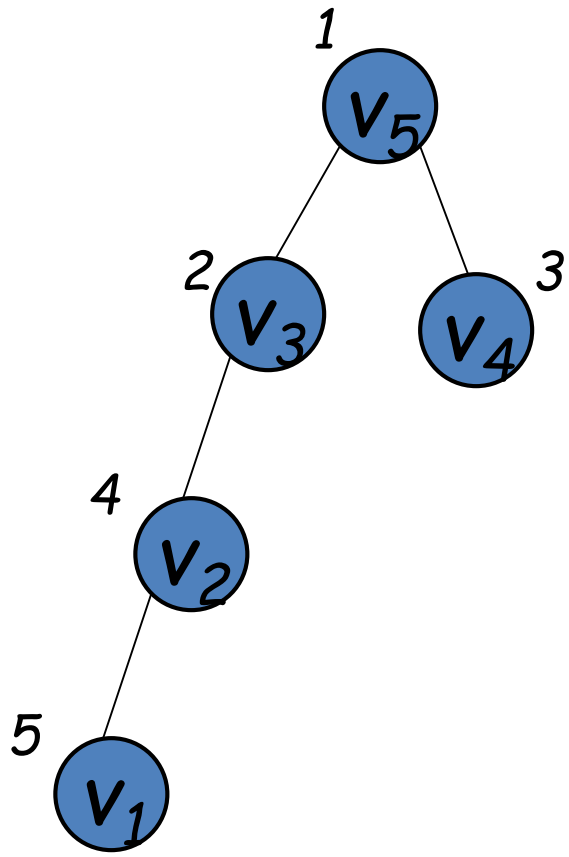
Basic Algorithm for BFS

```
Algorithm bfs(v)
    q.createQueue();
    q.enqueue(v);
    mark v as visited;
    while(!q.isEmpty()) {
       w = q.dequeue();
       for (each unvisited node u adjacent to w) {
         q.enqueue(u);
         mark u as visited;
       }
    }
```

BFS Illustration

- Start from v_5



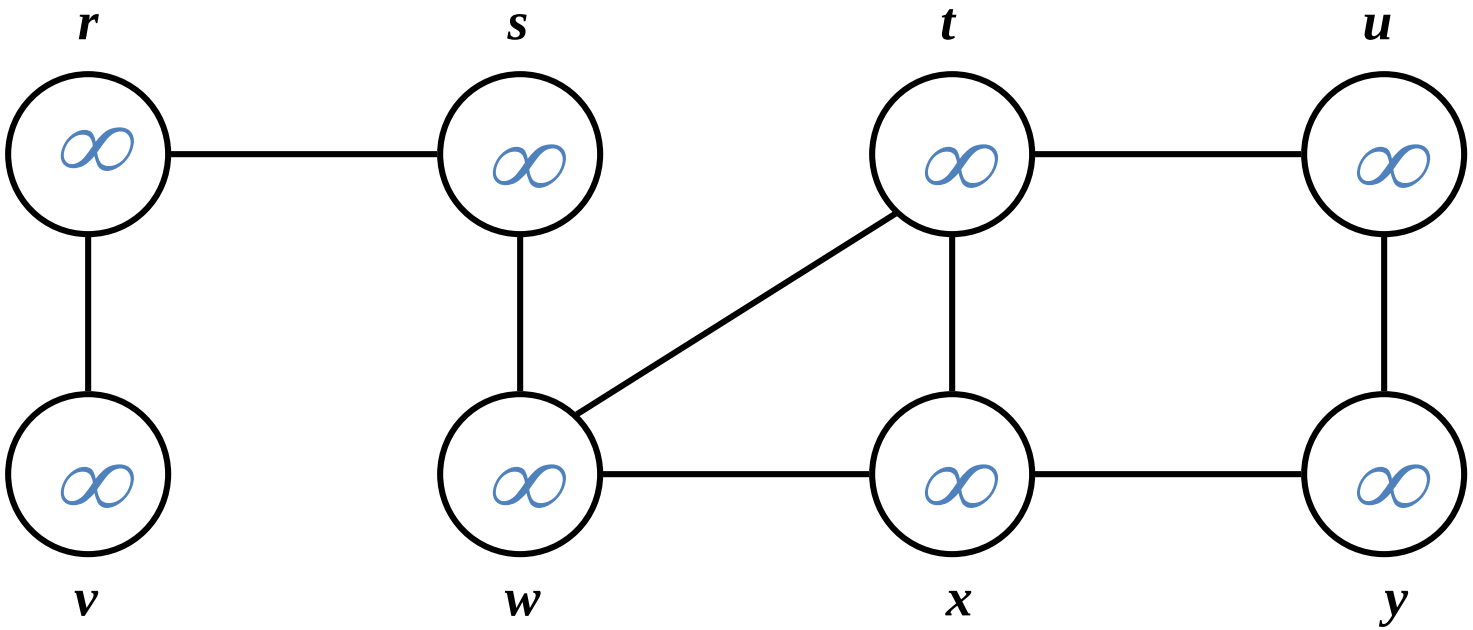


Visit	Queue (front to rear)
v_5	v_5
	empty
v_3	v_3
v_4	v_3, v_4
	v_4
v_2	v_4, v_2
	v_2
	empty
v_1	v_1
	empty

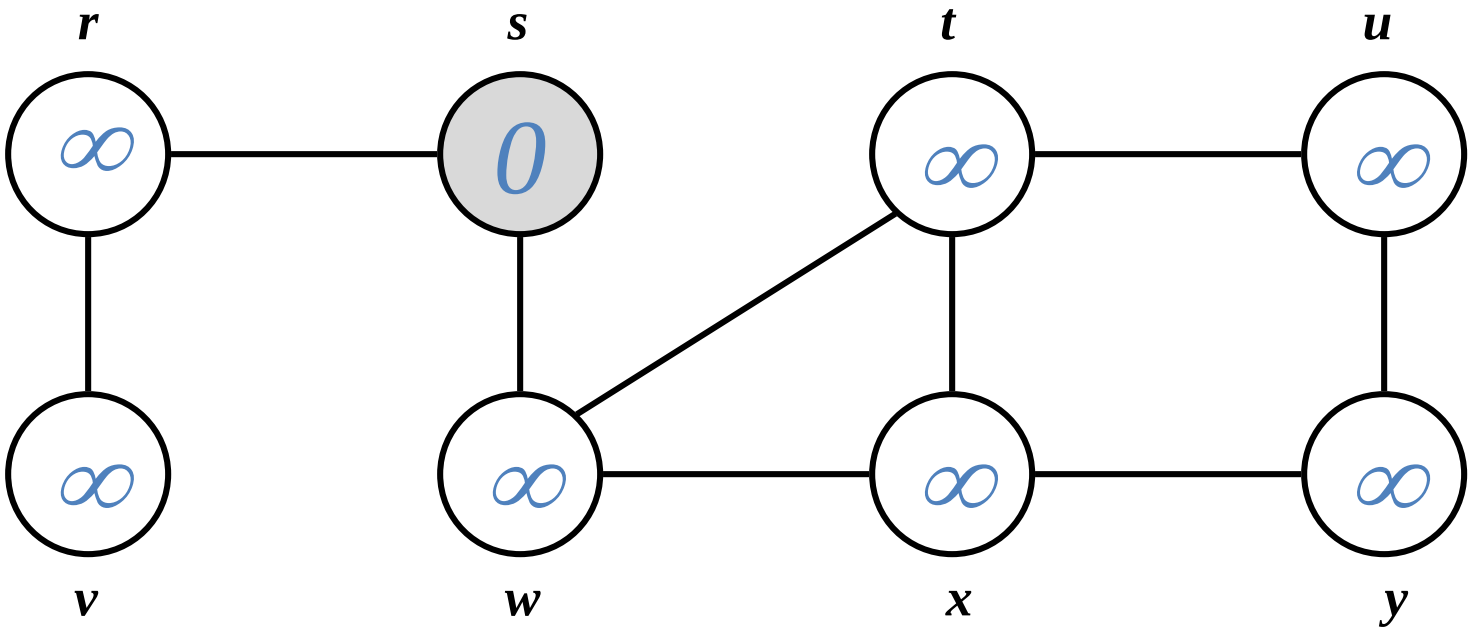
Breadth-First Search Algorithm

```
BFS(G, s) {
    initialize vertices;
    Q = {s};         // Q is a queue; initialize to s
    while (Q not empty) {
        u = Dequeue(Q);
        for each v ∈ u->adj {
            if (v->color == WHITE)
                v->color = GREY;
                v->p = u;
                Enqueue(Q, v);
        }
        u->color = BLACK;
    }
}
```

Breadth-First Search: Example

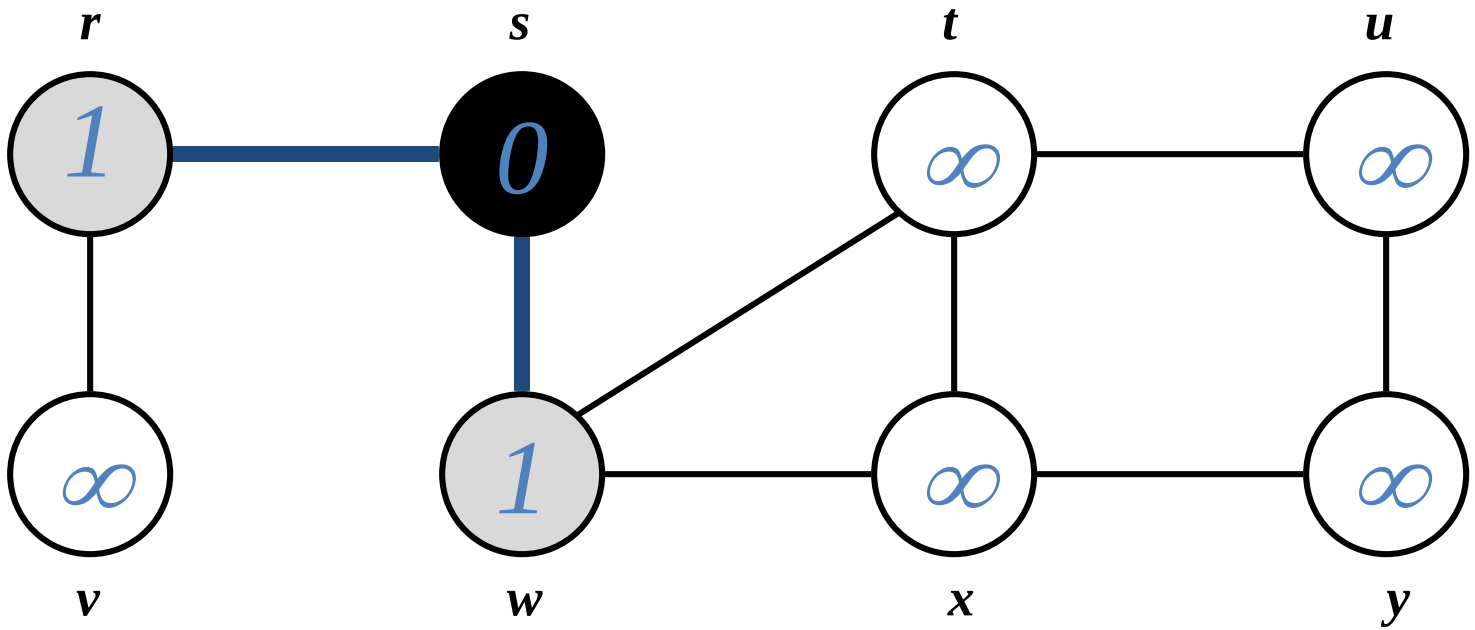


Breadth-First Search: Example



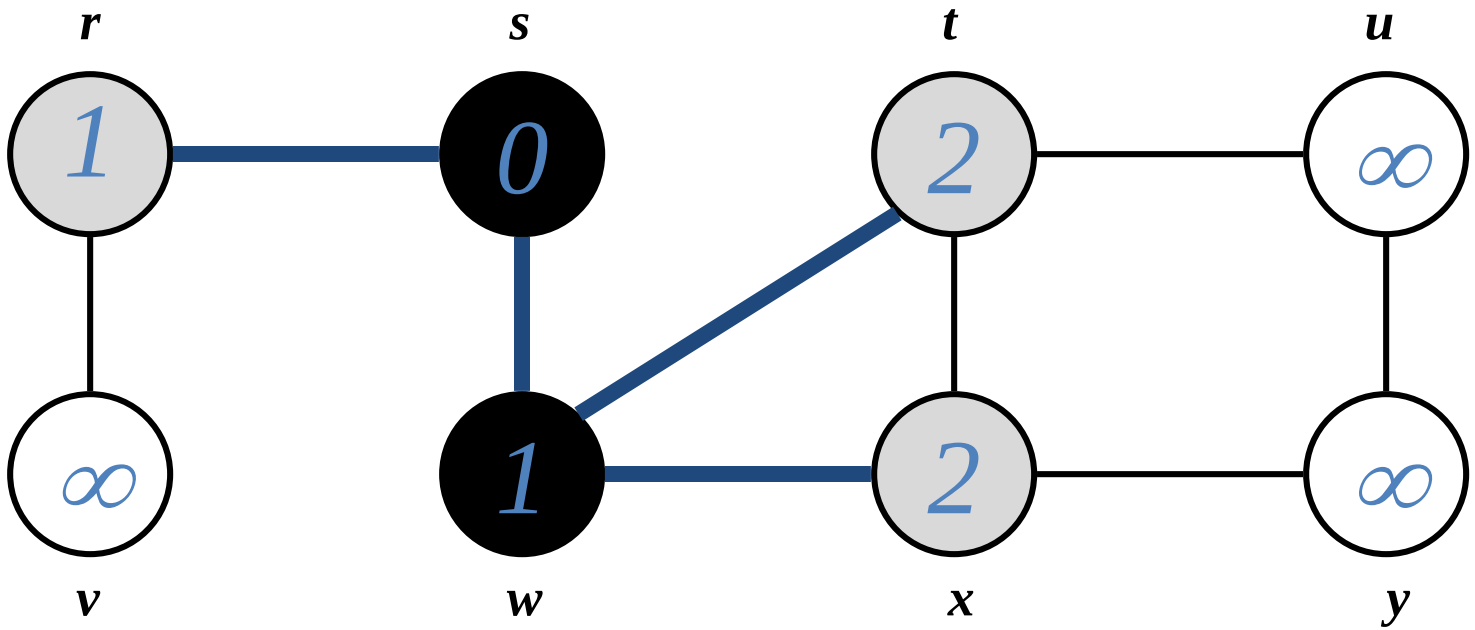
Q: | *s* |

Breadth-First Search: Example



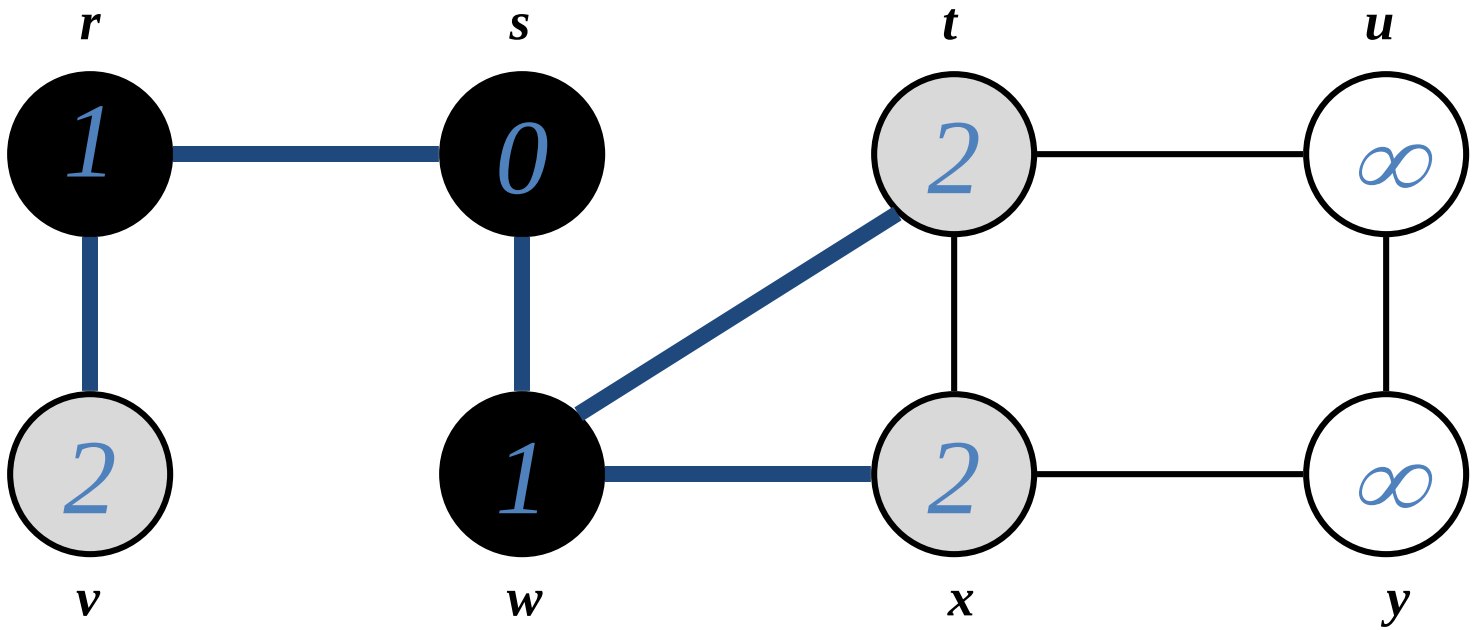
Q: | *w* | *r* |

Breadth-First Search: Example



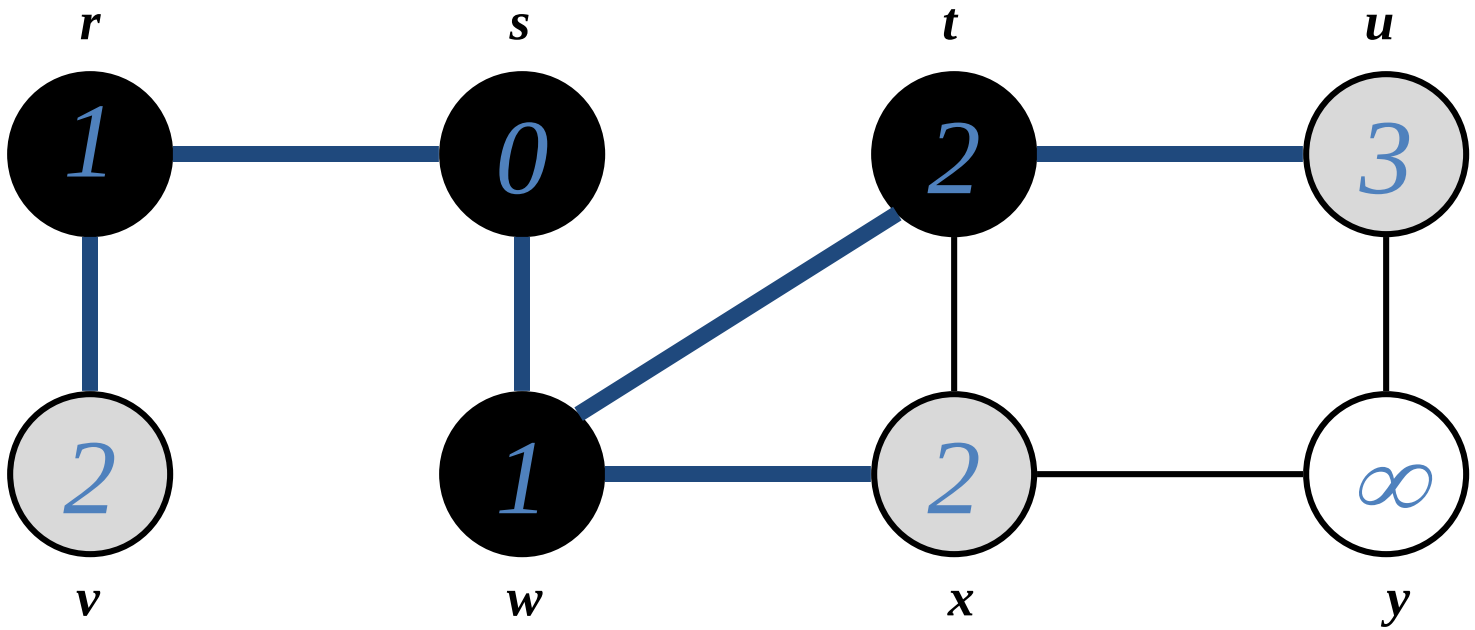
Q: | *r* | *t* | *x* |

Breadth-First Search: Example



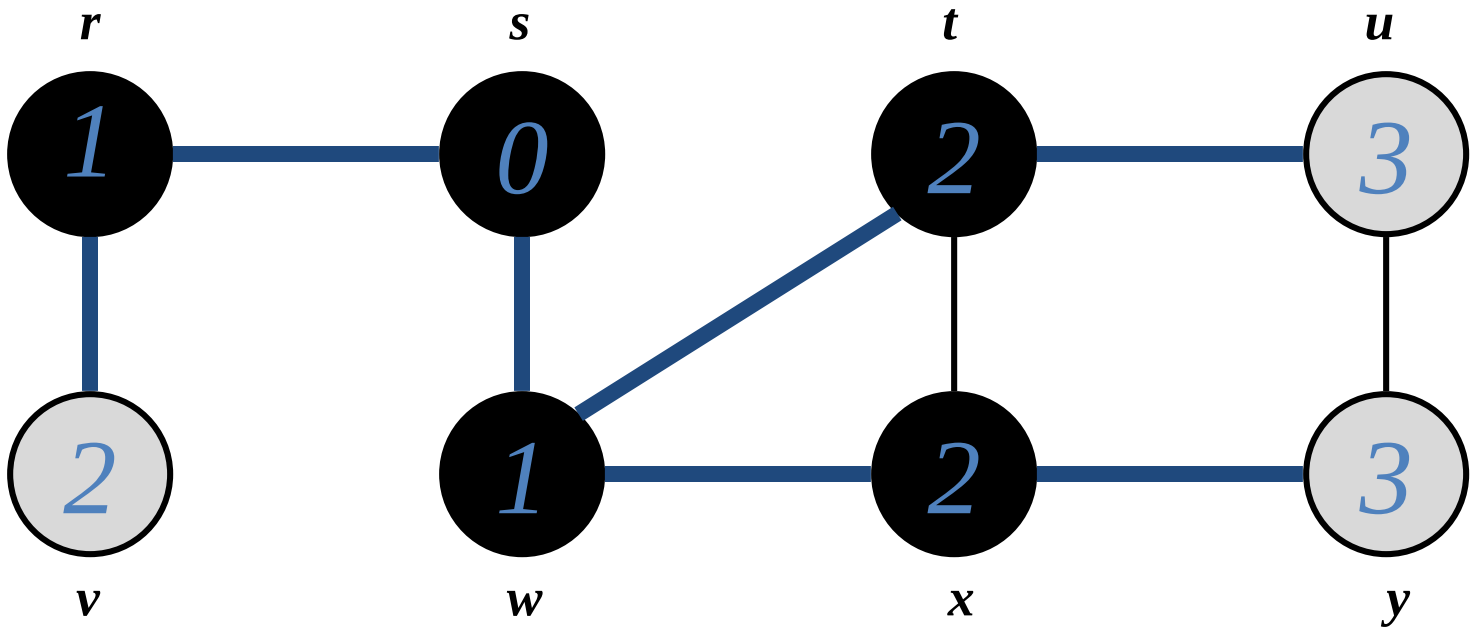
Q: | *t* | *x* | *v* |

Breadth-First Search: Example



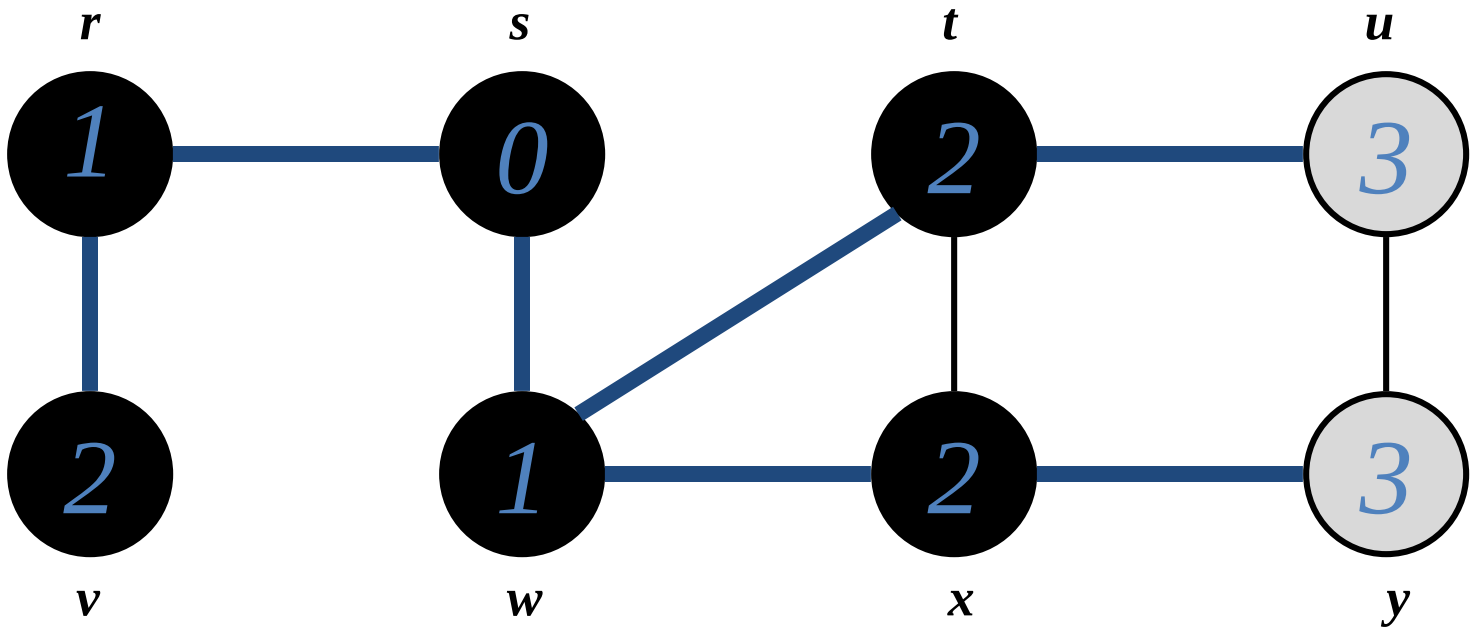
Q: | *x* | *v* | *u* |

Breadth-First Search: Example



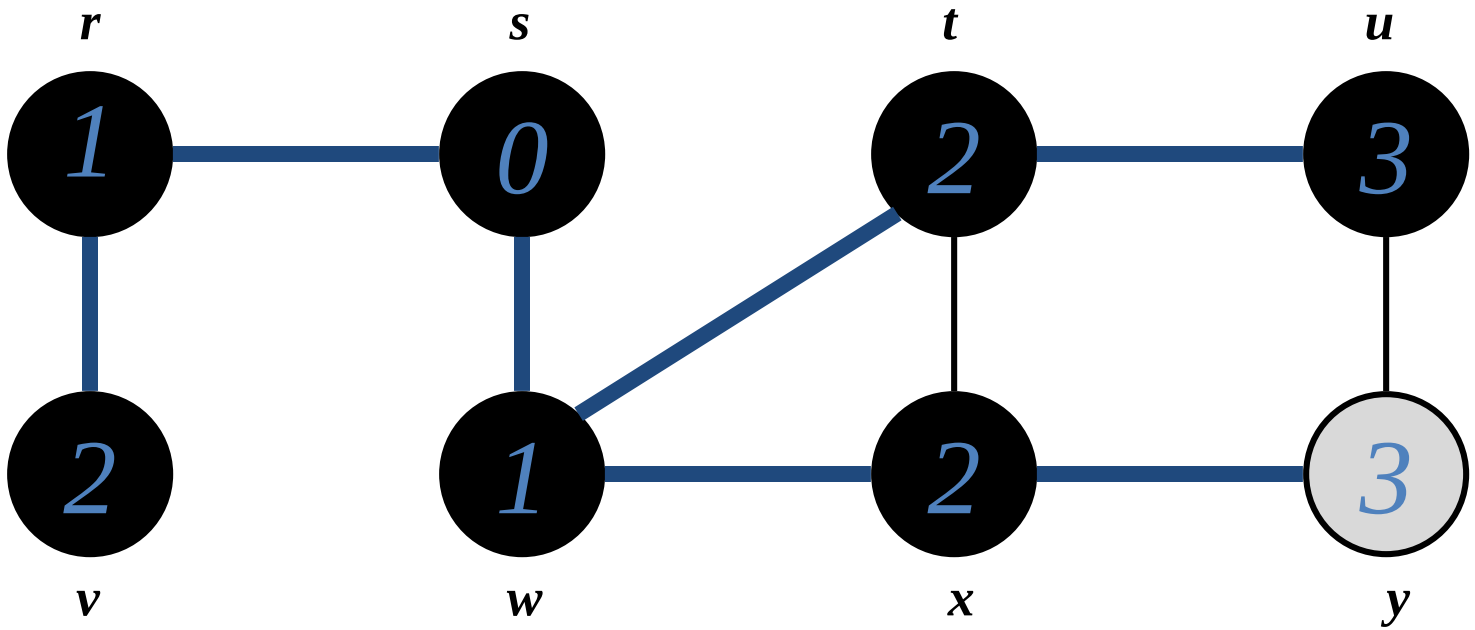
Q: | *v* | *u* | *y* |

Breadth-First Search: Example



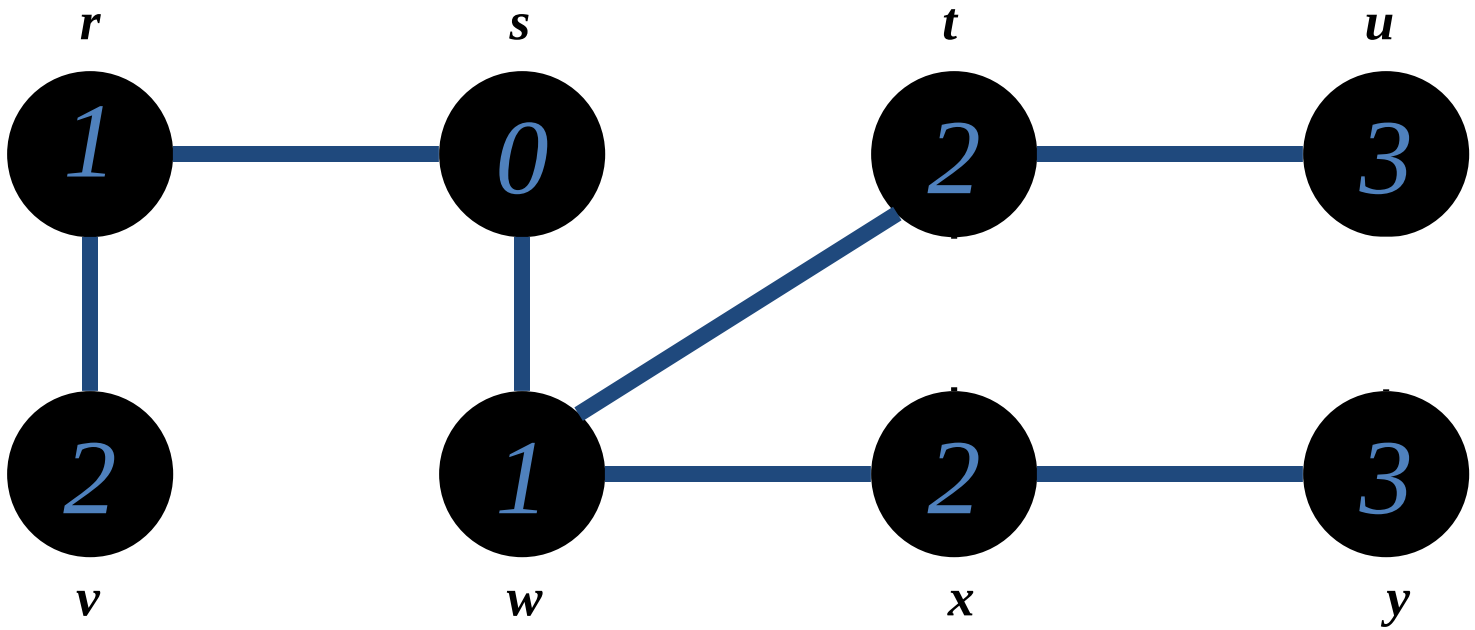
Q: | u | y |

Breadth-First Search: Example



Q: y

Breadth-First Search: Example



Q: Ø

Analysis of DFS and BFS

- Setting a vertex/edge label takes $O(1)$ time
- Each vertex is labeled twice
  - once as UNVISITED
  - once as VISITED
- Each edge is labeled twice
  - once as UNVISITED
  - once as VISITED
- DFS and BFS run in $O(n + m)$ complexity, where n is the no. of vertices and m is the no. of edges.